

Bases de Datos II

Tarea Corta 2

Equipo de trabajo:

Granados Retana, Diego - 2022158363

Granados Retana, Daniel - 2022104692

Mora Montes, Diego - 2022104866

Gutierrez Conejo, Eduardo - 2019073558

Cardona Quesada, Jose Ricardo - 2021022613

Manual de instalación

A continuación, se presenta un manual de instalación para distintos motores de bases de datos junto con respectivos códigos para implementar backups y restaurar datos a través de una cuenta AWS.

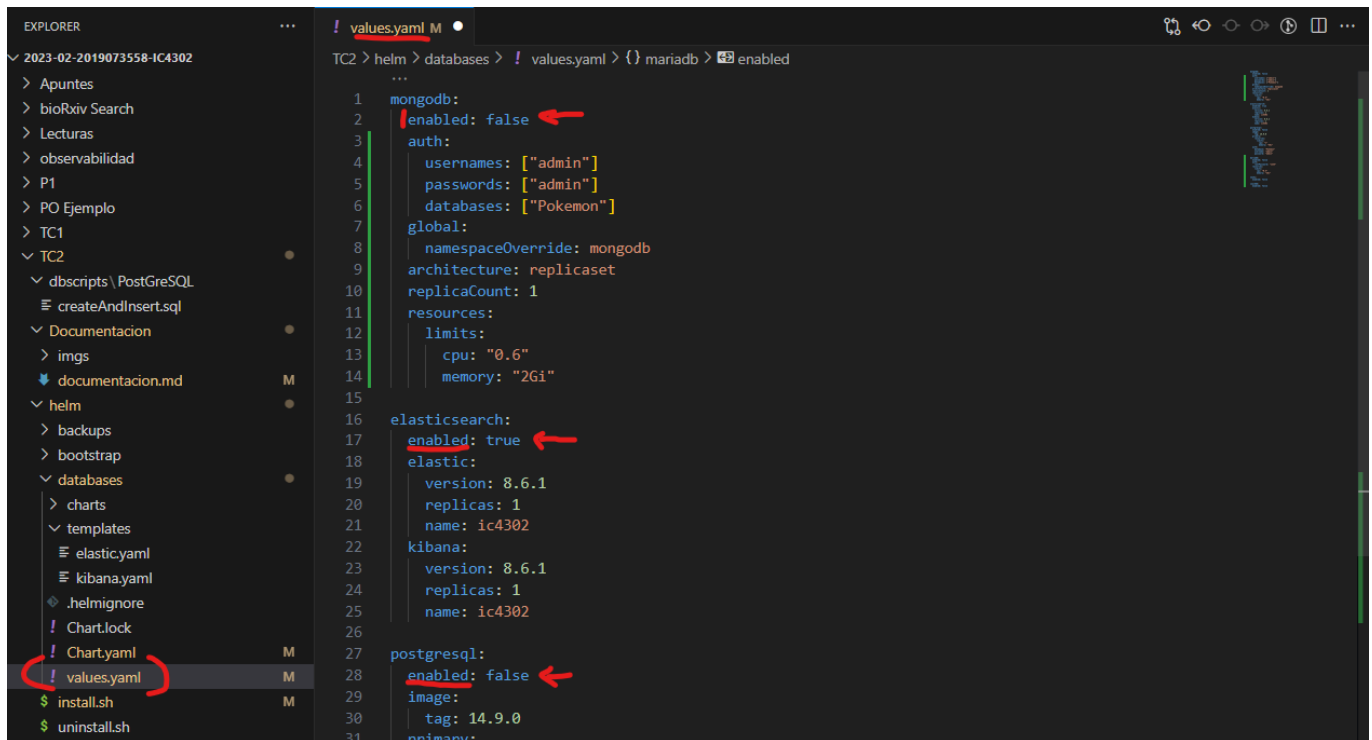
Paso 1. Selección del Motor de Bases de datos a probar

Dentro de la carpeta para la TC2 se incluyen varios motores de bases de datos, la lista de todos estos es:

- MariaDB
- PostgreSQL
- Elasticsearch
- MongoDB
- Neo4J
- CouchDB

Una vez se haya seleccionado la base de datos para la que se desea probar backups y restauración se hace lo siguiente:

Se ingresa al path `TC2\helm\databases\values.yaml` donde se va a cambiar ciertos parámetros para hacer backups solo en la base deseada e iniciar solo la base deseada. Por cada base de datos saldrá el siguiente parámetro:



```
1  mongodb:
2    enabled: false
3    auth:
4      usernames: ["admin"]
5      passwords: ["admin"]
6      databases: ["Pokemon"]
7  global:
8    namespaceOverride: mongodb
9  architecture: replicaset
10 replicaCount: 1
11 resources:
12   limits:
13     cpu: "0.6"
14     memory: "2Gi"
15
16 elasticsearch:
17   enabled: true
18   elastic:
19     version: 8.6.1
20     replicas: 1
21     name: ic4302
22   kibana:
23     version: 8.6.1
24     replicas: 1
25     name: ic4302
26
27 postgresql:
28   enabled: false
29   image:
30     tag: 14.9.0
31   primary:
```

enabled hace referencia a si se quiere inicializar o no ese motor de base de datos, la recomendación es no correr más de uno al mismo tiempo.

En caso de querer correr ese motor cambiar `enabled: true` por la versión actual, y al resto de motores ingresar `enabled: false` para apagarlos

Una vez hecho esto se debe ingresar a `TC2\helm\backups\values.yaml`, y hacer lo mismo con las bases de datos que aparecen.

IMPORTANTE: En caso de querer ejecutar backups a través de elastic **NO es necesario** ingresar a `TC2\helm\backups\values.yaml` a habilitar la base también. Sin embargo, sí es necesario acceder al archivo de values.yaml en el folder de `TC2\helm\bootstrap\values.yaml` y poner `enabled: true` en *elasticsearch*. Esto habilita el *elastic operator* en la instalación del helm chart de *bootstrap*. Este es importante para correr elasticsearch.

Finalmente, antes de ejecutar el código, se debe decidir si se desea realizar un backup o un restore en la base de datos. Esto se escoge en el archivo `TC2\helm\backups\values.yaml`, en el campo de *config.type*. Si queremos hacer un backup, simplemente ponemos `type: backup` y si queremos hacer un restore, ponemos `type: restore`. Cabe destacar que cuando queremos hacer un restore, debemos especificar el nombre del

backup que queremos restaurar de los que están en aws. Esto se especifica en el campo *name*.

```
31  postgresql:
32      enabled: false
33  config:
34      namespace: default
35      host: databases-postgresql.default.svc.cluster.local
36      db: "Pokemon"
37      user: "admin"
38      password: "admin"
39      port: "5432"
40      bucketName: tec-ic4302-02-2023
41      path: 2019073558/postgresql
42      maxBackups: 3
43      name: Pokemon-202310211634.sql
44      schedule: "0 */12 * * *"
45      diskSize: 2
46      storageClass: hostpath
47      provider: aws
48      type: restore
```

Paso 2. Ejecución del Código

Una vez se haya escogido la base, el siguiente paso es ejecutar los comandos para levantarla con sus respectivos parámetros, para esto ingresamos a cualquier terminal desde la cuál se puedan ejecutar comandos Linux. Una vez aquí abriremos la carpeta desde la terminal:

```
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302# cd TC2
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2# cd helm
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm#
```

Una vez en la carpeta donde hayamos descargado el proyecto se insertarán los siguientes comandos:

```
cd TC2 cd helm
```

Una vez aquí la terminal se encuentra en la carpeta donde se pueden ejecutar los archivos para la instalación y desinstalación de las bases de datos.

Para Instalar la base deseada con los parámetros especificados

En la terminal, primero se recomienda ejecutar:

```
dos2unix install.sh
```

Seguido de esto ejecutamos el comando: `./install.sh`

Una vez hecho esto la terminal comenzará a instalar todos los helm charts necesarios para la ejecución correcta del programa:

```

root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm# dos2unix install.sh
dos2unix: converting file install.sh to Unix format...
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm# ./install.sh
2023/10/20 11:09:43.130990 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:09:43.135607 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number
of tail fields, expected 3 but found 4
Saving 1 charts
Downloading eck-operator from repo https://helm.elastic.co
Deleting outdated charts
2023/10/20 11:09:45.428497 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:09:45.433184 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number
of tail fields, expected 3 but found 4
Release "bootstrap" does not exist. Installing it now.
NAME: bootstrap
LAST DEPLOYED: Fri Oct 20 11:09:45 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None

```

Una vez este código se verá lo siguiente:

```

Release "databases" does not exist. Installing it now.
NAME: databases
LAST DEPLOYED: Fri Oct 20 11:30:02 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
2023/10/20 11:31:02.794566 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:31:02.802229 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number of tail f
lds, expected 3 but found 4
2023/10/20 11:31:03.243505 system_key.go:129: cannot determine nfs usage in generateSystemKey: cannot parse mountinfo: incorrect number of tail fields, expected 3 but found 4
2023/10/20 11:31:03.248048 cmd_run.go:1046: WARNING: cannot create user data directory: cannot determine SELinux status: failed to obtain SELinux mount path: incorrect number of tail f
lds, expected 3 but found 4
Release "backups" does not exist. Installing it now.
NAME: backups
LAST DEPLOYED: Fri Oct 20 11:31:03 2023
NAMESPACE: default
STATUS: deployed
REVISION: 1
TEST SUITE: None
root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/TC2/helm#

```

Este código habilita la base deseada para las pruebas que se quieran ejecutar con ella. Cada una de las bases es distinta.

Ejecución de Bases y pruebas

IMPORTANTE

Para poder verificar si los archivos correspondientes a una de las bases se encuentran en el bucket, se puede hacer lo siguiente:

En la terminal, con el proyecto abierto, nos aseguramos de volver a la carpeta TC2 y en caso de estar adentro de Helm, hay que regresar. Para esto, se ejecuta el comando `cd ..` hasta regresar a la dirección del TC2.

Una vez en esta dirección, se ejecutan los comandos:

```
cd utils
```

```
dos2unix install.sh
```

```
./install.sh
```

Este código va a instalar un pod en la red de Kubernetes donde se pueden ejecutar comandos para verificar si el bucket tiene archivos en la dirección que se quiere revisar. Al finalizar se podrá ver lo siguiente:

```

root@RENE-LAPTOP-G7-7588:/mnt/d/OneDrive/Desktop/PrograJose/2023-02-2019073558-IC4302/tc2/utills# ./install.sh
Using default tag: latest
latest: Pulling from aws-cli/aws-cli
daa81221dec1: Already exists
ac9dcbe20ae: Pull complete
0e4f469ee5ea: Pull complete
0f0d26aefdf3: Pull complete
382122d245f9: Pull complete
Digest: sha256:e668c96d1df44d3796e4faf00121e9f38fdfed8c9b1439eff0aa6a473c694417
Status: Downloaded newer image for public.ecr.aws/aws-cli/aws-cli:latest
public.ecr.aws/aws-cli/aws-cli:latest
pod/debug-pod created
bash-4.2#

```

Aquí se pueden ingresar comandos para revisar el contenido de los buckets en aws. El que lista los archivos que están en una dirección dada es: `aws s3 ls s3://tec-ic4302-02-2023/[ruta]` En nuestro caso, nuestros backups se están almacenando a partir de la ruta /2019073558, por lo que la instrucción que usaríamos comenzaría con: `aws s3 ls s3://tec-ic4302-02-2023/2019073558/`

Elasticsearch con Kibana

A continuación, se muestra una guía para hacer backups y restores en la base Elasticsearch a través de su interfaz Kibana. Para este paso se asume que ya se habilitó la base y se ejecutó el script de instalación desde la terminal.

Lo primero que se debe hacer es tener instalada una aplicación para poder visualizar el kubernetes deployment y fácilmente obtener la información importante de los pods.

Para esta funcionalidad se recomienda hacer uso de **LENS**, el resto de esta guía se basará en el uso de esta aplicación para conectar los pods.

Desde Lens lo primero que haremos será entrar el cluster de Docker Desktop y nos iremos a la sección de **PODS**, una vez aquí entraremos al siguiente pod:

The screenshot shows the Kubernetes Lens application interface. On the left, the 'Pods' tab is selected, displaying a table with 3 items. The table has columns for Name, Namespace, Container, CPU, and Memory. The pod 'ic4302-kb-5f796466db-vcnrt' is highlighted. On the right, the details for this pod are shown. The 'Containers' section lists 'kibana'. Below this, the 'Ports' section shows a link 'https: 5601/TCP' with a red arrow pointing to it, and a 'Stop/Remove' button. The 'Environment' section is currently empty.

Name	Namespace	Container	CPU	Memory
elastic-operator	default		N/A	N/A
ic4302-es-defal	default		N/A	N/A
ic4302-kb-5f796466db-vcnrt	default		N/A	N/A

Pod: ic4302-kb-5f796466db-vcnrt

Containers

- kibana
 - CPU
 - Memory
 - Filesystem

Metrics not available at the moment

Status: running, ready

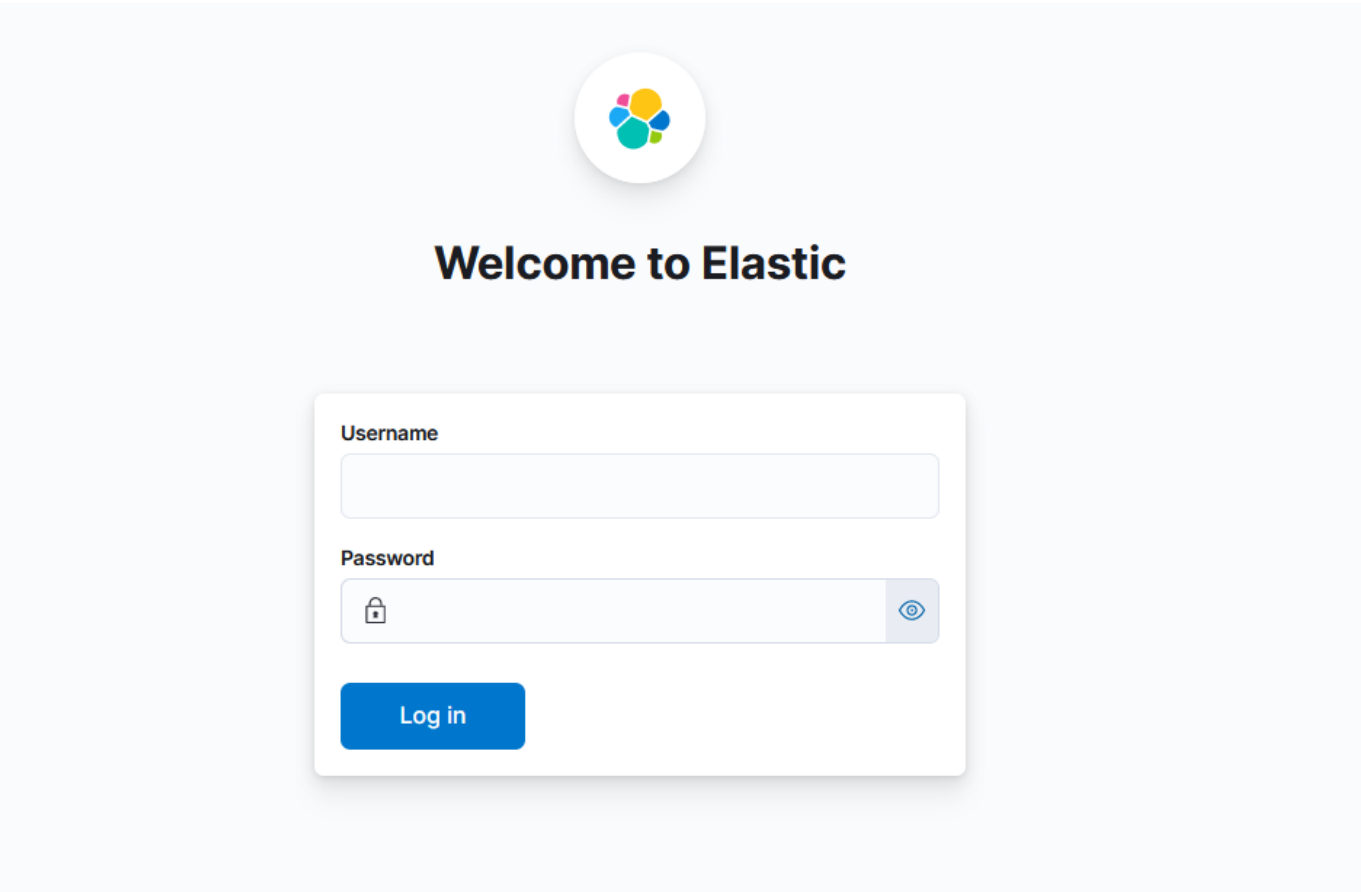
Image: docker.elastic.co/kibana/kibana:8.6.1

Ports: [https: 5601/TCP](https://5601/TCP) Stop/Remove

Environment: —

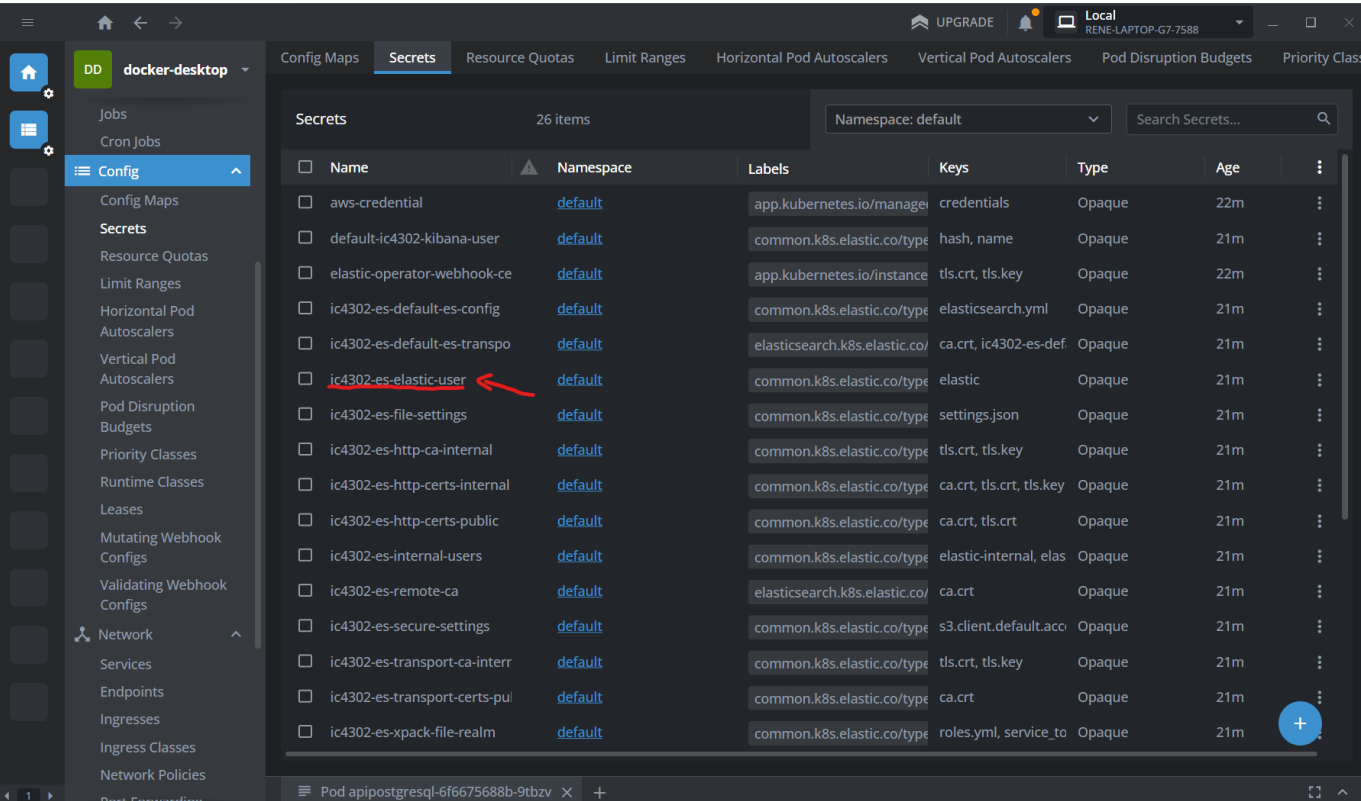
Al abrir el pod se debe buscar la sección señalada a la derecha, esta contiene un link que nos permitirá acceder de forma local a la interfaz Kibana para interactuar con la base de datos elasticsearch.

Una vez se haya hecho forward del port y se haya ingresado al sitio aparecerá lo siguiente:

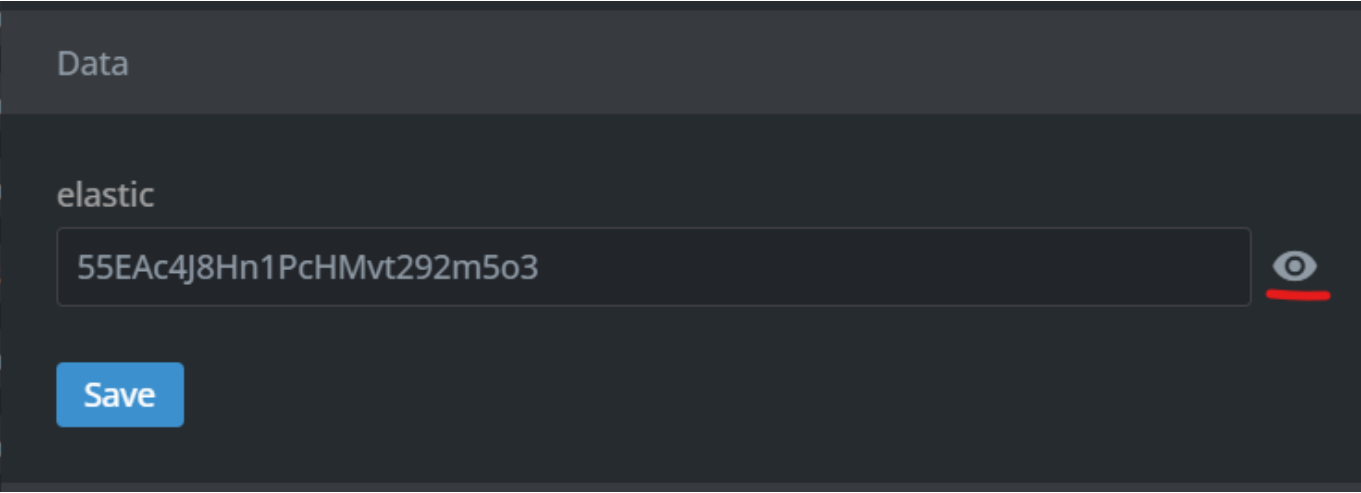


Se solicita un usuario y una contraseña para poder ingresar al servicio Kibana, para obtener estos datos volvemos a **LENS** y buscamos la sección:

Config/Secrets, una vez aquí abrimos el siguiente secret:

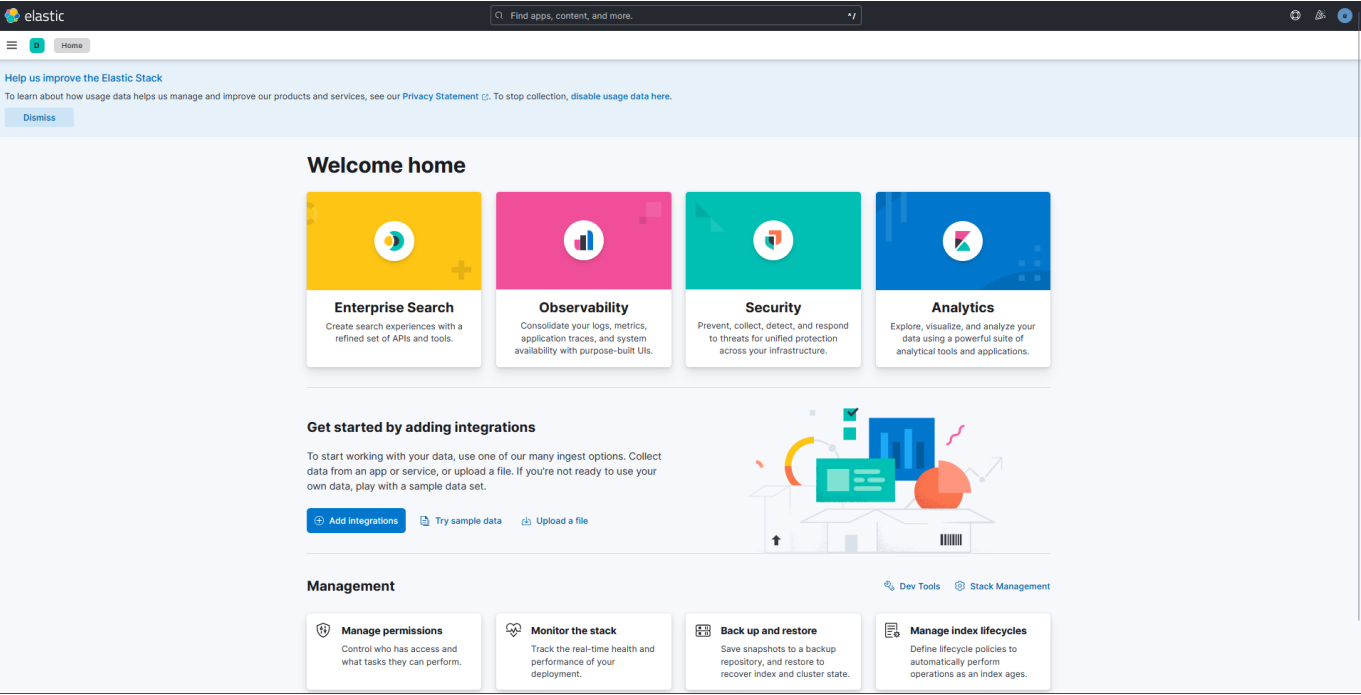


Una vez aquí buscamos los siguientes datos:

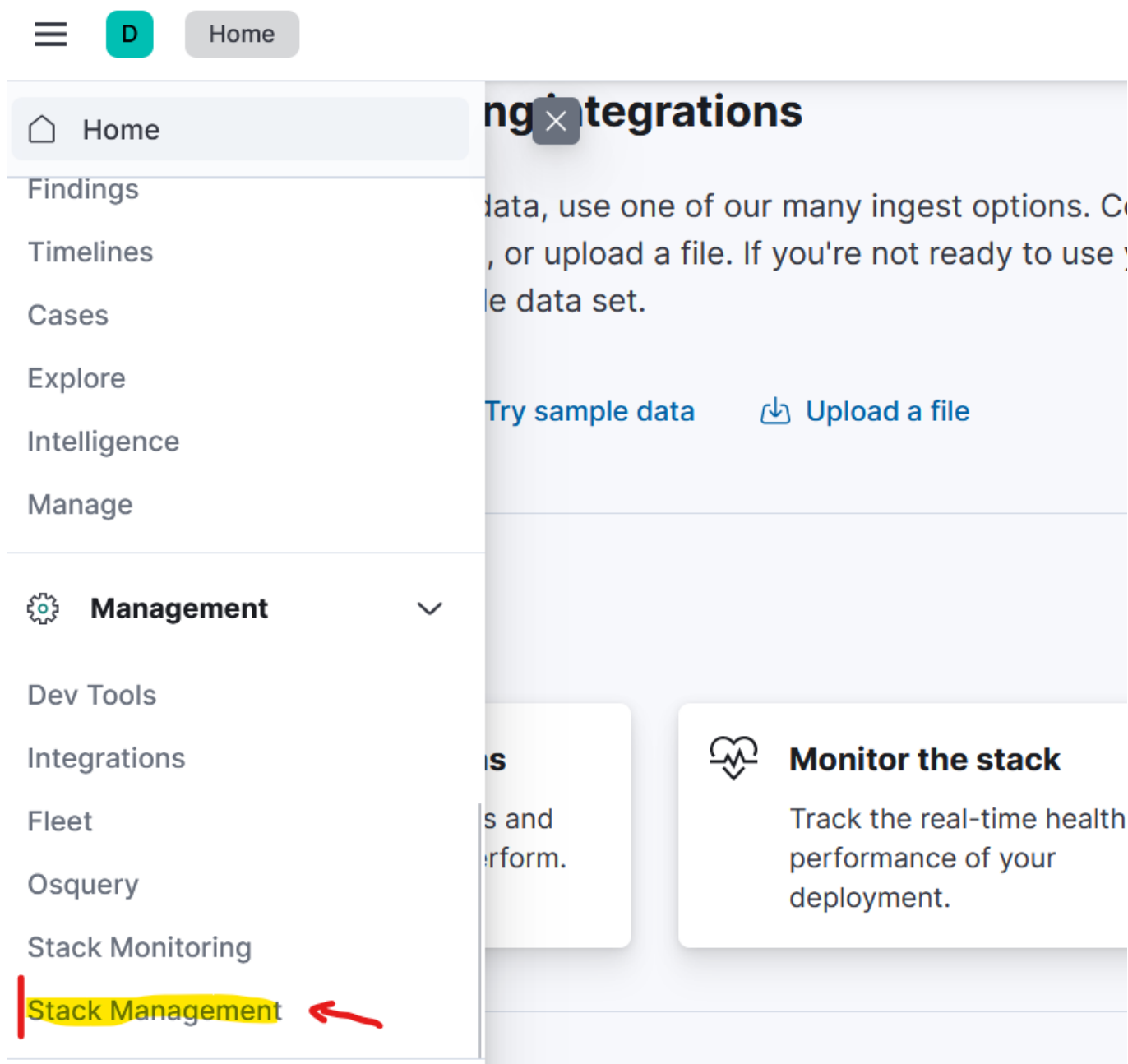


Estos corresponden a mi usuario y contraseña de elasticsearch. Por lo tanto, copio estas credenciales y las pongo en la interfaz. El usuario será elastic y la contraseña el otro valor. Hay que hacer click sobre el ojo para descryptar la contraseña.

Después de ingresar las credenciales correctamente saldrá una imagen similar a esta:

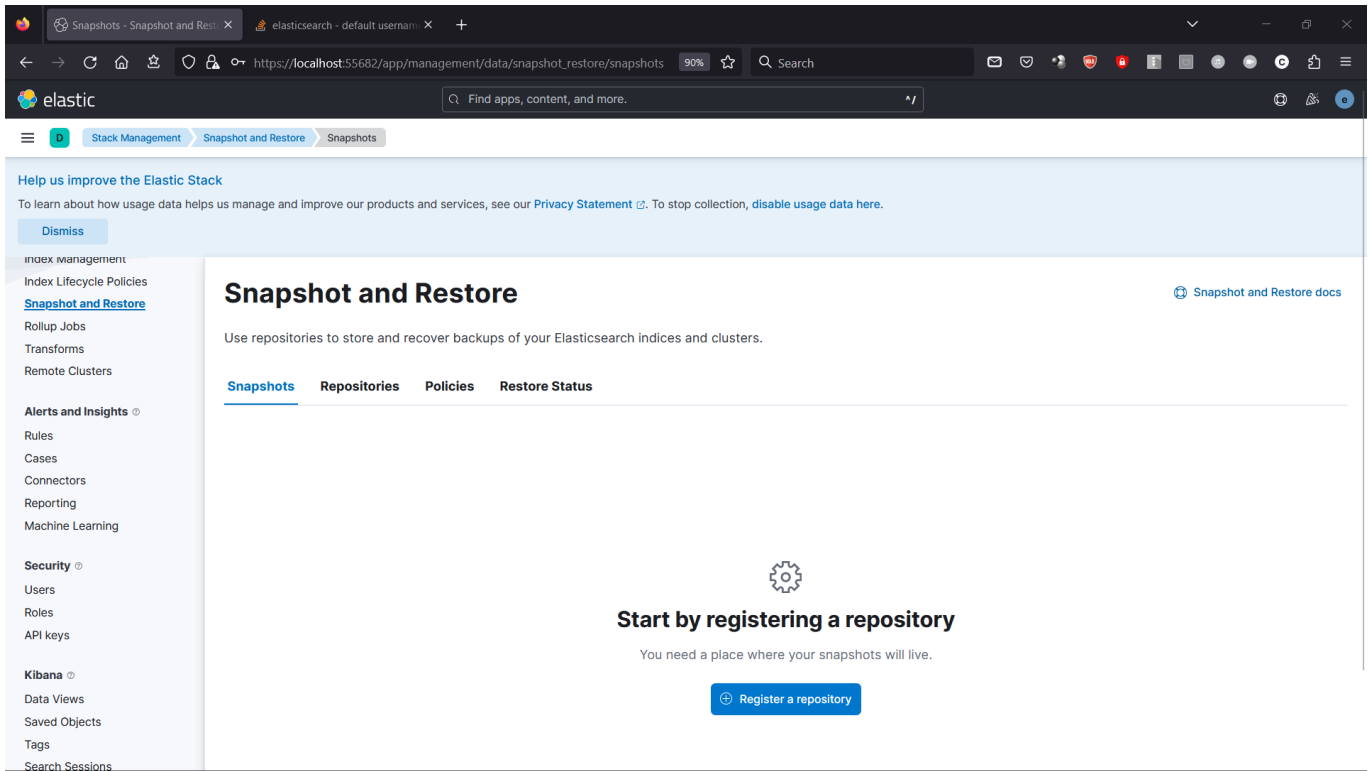


En esta pantalla buscaremos la opcion stack management e ingresaremos



Una vez en esta interfaz, se busca la sección con el nombre **Snapshot and Restore**, en elastic los backups son llamados de esta forma.

Para restaurar o guardar backups lo primero es definir la dirección donde se van a guardar/obtener. Para esto se va a agregar el repositorio donde vamos a manejar nuestros backups de elastic, este hace referencia a la dirección del bucket de AWS y el cliente que vamos a usar para guardar los datos correspondientes a los snapshots.



Una vez aquí se selecciona **Register a repository** para crearlo. Una vez adentro:

Seleccionamos el tipo de repositorio, en este caso debe de ser AWS S3 y se le pone el nombre que quiera al repositorio.

Repository name

A unique name for the repository.

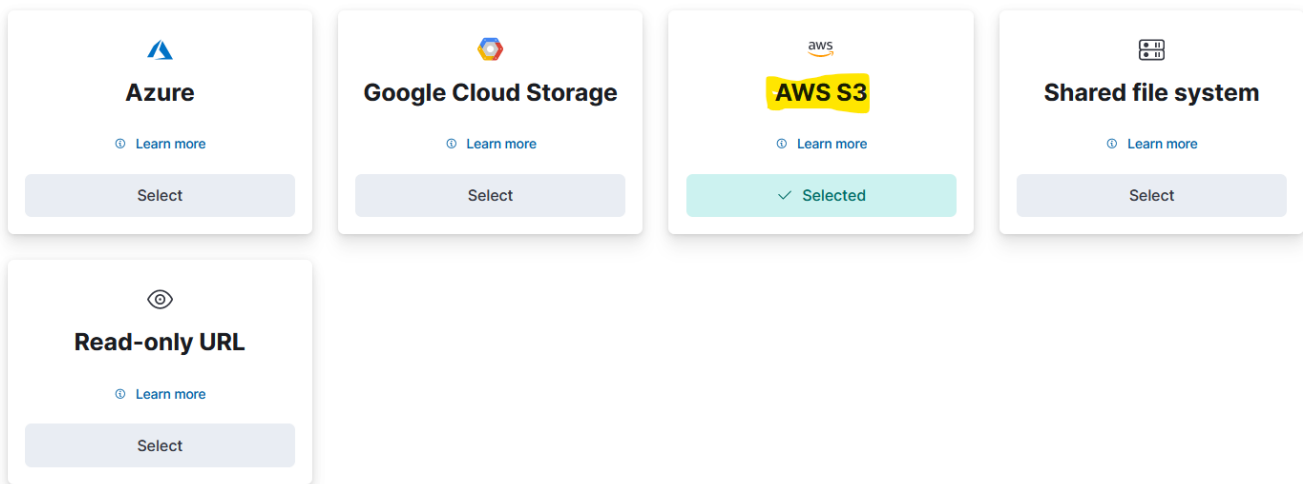
Name

BackupsElastic

Repository name is required.

Repository type

Storage location for your snapshots. [Learn more about repository types.](#)



En la siguiente pantalla se solicitan los datos que le queremos poner al repositorio. En esta sección los únicos campos relevantes son:

Register repository

'BackupsElastic' settings

[AWS S3 repository docs](#)**Client**

The name of the AWS S3 client.

Client

default

Bucket

The name of the AWS S3 bucket to use for snapshots.

Bucket (required)

tec-ic4302-02-2023

Base path

The bucket path to the repository data.

Base path

2021022613/elastic

Snapshot compression

Compresses the index mapping and setting files for snapshots. Data files are not compressed.



Compress snapshots

En el campo Client ingresamos: **default**

En el campo bucket ingresamos: **tec-ic4302-02-2023**

En el campo Base path se debe ingresar: **2019073558/elastic**

Es importante anotar este base path para posteriormente usarlo al definir la política para los backups. El resto de campos para el repositorio se dejan por default y se finaliza la creación de este. El resultado de este proceso se verá similar a esto:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

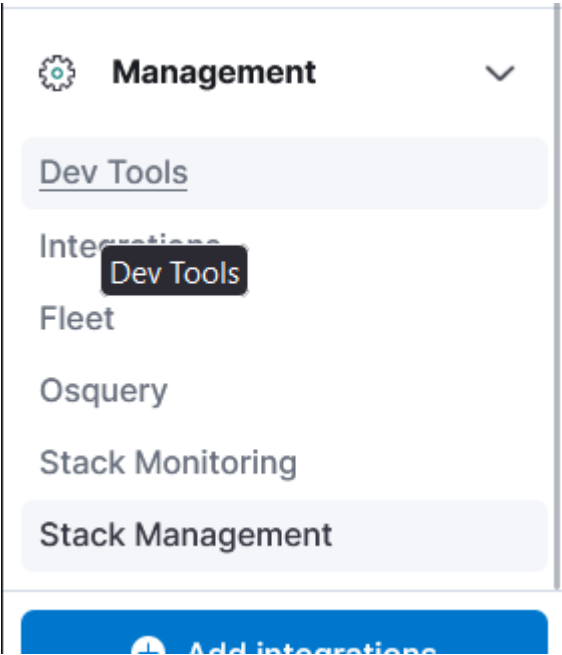
Snapshots **Repositories** Policies Restore Status

Search...		Type ▾	Reload	Register repository
☐ Name ↑	Type	Actions		
☐ BackupsElastic	AWS S3	Edit	Delete	

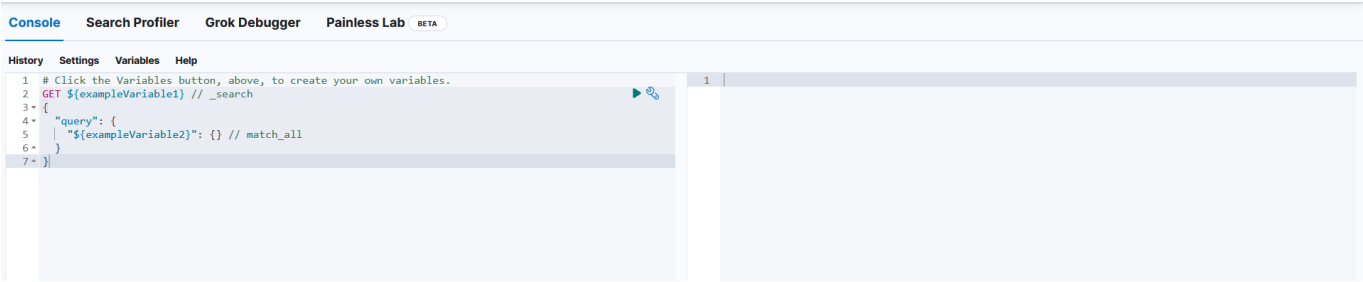
Para hacer pruebas

Cabe aclarar que el repo será adonde guardemos los backups, pero aún no se ha creado ninguna política para crear backups o se han creado índices para los cuáles probar esto. Por lo tanto antes de seguir con el proceso de backups se van a insertar índices en la base elasticsearch manualmente, posteriormente se mostrará como hacer un backup con esta. Finalmente se borrarán los datos y se tratará de restaurarlos usando el snapshot.

Primero, para ingresar los datos a la base es necesario volver al menú principal de la interfaz de Kibana, una vez aquí buscamos la siguiente sección:



Ingresamos a Dev Tools, y se presentará la siguiente interfaz:



Aqui escribimos los siguientes comandos:

PUT /indice-backup

POST /indice-backup/_doc { "titulo": "Prueba Backups", "contenido": "Este documento es una prueba de backups", "integrantes": ["Granados Retana Diego", "Granados Retana Daniel","Mora Montes Diego","Gutierrez Conejo Eduardo","Cardona Quesada Jose Ricardo"] }

GET /indice-backup/_search { "query": { "match": { "titulo": "Prueba Backups" } } }



Una vez hecho esto se insertaron datos y un índice para el cuál podemos hacer backups

Snapshots

Para guardar snapshots de nuestra base completa o un índice en ella hay dos maneras, se puede hacer manualmente a través de comandos o automáticamente definiendo una política de guardado.

Método Manual

Para hacer un backup manualmente, desde la misma interfaz de Dev Tools se puede ingresar un comando como este:

```
PUT /_snapshot/ [NOMBRE_REPOSITORIO] / [NOMBRE_SNAPSHOT] { "indices": "[NOMBRE INDICE]", "ignore_unavailable": true, "include_global_state": false }
```

Donde se reemplaza la dirección con el nombre del índice al que queremos tomar un snapshot. En este caso para el ejemplo se usara el siguiente:

```
PUT /_snapshot/BackupsElastic/indice-backup_20231020 { "indices": "indice-backup", "ignore_unavailable": true, "include_global_state": false }
```

Para revisar que este snapshot se encuentra disponible en el repositorio y en AWS se puede revisar de dos maneras:

La primera opción es ingresar a la sección de Stack Management y Snapshot-Restore dentro de la interfaz de Kibana. Una vez aqui se verá lo siguiente:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

Snapshots

Repositories

Policies

Restore Status

Search...

Repository

Reload

☐	Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration	Actions
☐	indice-backup_20231020	BackupsElastic	1	1	0	Oct 20, 2023 5:33 PM CST	3s	

Rows per page: 20

< 1 >

Si se logro crear correctamente el snapshot, va a aparecer en esta sección. La segunda opción es a través de el debug-pod, como se menciona en la sección IMPORTANTE, previo a esta. Al utilizar el debug pod se puede observar el índice recién almacenado

```
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2019073558/elastic/
PRE indices/
PRE tests-52XAchX2SnePp29GmISIRw/
2023-10-20 23:31:51 607 index-0
2023-10-20 23:31:51 8 index.latest
2023-10-20 23:31:50 212 meta-ijst37f1RPeqwsVBQeJ1ew.dat
2023-10-20 23:31:50 320 snap-ijst37f1RPeqwsVBQeJ1ew.dat
```

Automático

El segundo método para hacer backups, es estableciendolos automáticamente a través de un policy, usando uno de estos, se pueden establecer la creación de backups cada cierto tiempo automáticamente, junto con un tiempo para el cual cuando se pase sean eliminados automáticamente también.

Para esto en la misma sección de Snapshots and Restore dentro de la Interfaz de Kibana se accede a la sección de Snapshot and Restore y una vez ahí se entra a Políticas/Create a Policy:

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

[Snapshots](#) [Repositories](#) [Policies](#) [Restore Status](#)



You don't have any snapshots yet

Run a Snapshot Lifecycle Policy to create a snapshot. You can also create a snapshot using [the Elasticsearch API](#).

[+ Create a policy](#)

Si se ingresa a esta Interfaz se le presenta al usuario con lo siguiente:

Logistics

[Logistics docs](#)

Policy name

A unique identifier for this policy.

Name

Policy-ic4302

Snapshot name

The name for the snapshots. A unique identifier is automatically added to each name.

Snapshot name

snapshot-elastic-{now/d}>

Supports date math expressions. [Learn more.](#)

Repository

The repository where you want to store the snapshots.

Repository

BackupsElastic

Schedule

The frequency at which to take the snapshots.

Frequency

Every hour

Minute

At 30

[Create cron expression](#)

[Next >](#)

[Cancel](#)

Aquí se solicitan los siguientes datos del usuario:

- **Policy Name:** Aquí se le pone un nombre a la política, este valor es a gusto del usuario
- **Snapshot Name:** En esta sección se le debe de poner un nombre a los snapshots que se van a almacenar, la plataforma automáticamente les agrega un identificador único al final
- **Repository:** Aquí se debe poner el nombre del repositorio previamente creado, si no se ingresa un repositorio válido no se creará la política

- **Schedule:** En esta parte se debe definir cada cuanto tiempo se quieren hacer backups de la plataforma, se puede definir si se quiere cada ciertas horas, dias, mes, etc.

Una vez se han ingresado los datos, se pasa a la siguiente página:

Snapshot settings [Snapshot docs](#)

Data streams and indices
To back up indices and data streams, manually select them or define index patterns to dynamically match them. ☒ All data streams and indices

Ignore unavailable indices
Ignores indices that are unavailable when taking the snapshot. Otherwise, the entire snapshot will fail. ☐ Ignore unavailable indices

Allow partial indices
Allows snapshots of indices with primary shards that are unavailable. Otherwise, the entire snapshot will fail. ☐ Allow partial indices

Include global state
Stores the global cluster state as part of the snapshot. ☒ Include global state

Include feature state
Includes the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security. ☒ Include feature state from
All features

[< Back](#) [Next >](#) [Cancel](#)

En esta sección no es necesario cambiar nada, es cuestión de dejarlo con sus configuraciones por default, se pasa a la siguiente página:

Create policy [Snapshot docs](#)

Snapshot retention (optional)

Expiration
The time to wait before deleting snapshots.

Snapshots to retain
The minimum and maximum number of snapshots to store for the policy.

[< Back](#) [Next >](#) [Cancel](#)

Esta es la sección final para crear un policy, aquí se puede definir **Si se quieren borrar los backups después de cierto tiempo, y si se quieren definir un mínimo a mantener**. A través de esta sección se puede configurar los puntos extra de la tarea, indicando la cantidad de días tras los que se quiere borrar los backups. Finalmente, le damos next y se va a crear el policy en esta sección:

Review policy

Summary Request

Logistics

Policy name

Policy-ic4302

Repository

BackupsElastic

Snapshot name

snapshot-elastic-{now/d}>

Schedule

0 30 * * * ?

Snapshot settings

Data streams and indices

All indices

Include global state

Yes

Allow partial indices

No

Ignore unavailable indices

No

Include feature state

All features

Snapshot retention

Delete after

3d

Min count

1

< Back

✓ Create policy

Cancel

Confirmamos que los datos están correctos y se le da crear,como resultado, si se hizo bien debe de aparecer lo siguiente:

×

SnapshotsPrueba-Elastic

Summary

History

Snapshots taken

Failures

Deleted

Deletion failures

0

0

0

0

General

Version

1

Snapshot name

<snapshot-prueba-elastic>

Schedule

0 0 * * * ?

Data streams and indices

indice-backup

Include global state

Yes

Allow partial shards

No

Last modified

Oct 21, 2023 12:00 PM CST

Repository

BackupsElastic

Next snapshot

Oct 21, 2023 1:00 PM CST

Ignore unavailable indices

No

Include feature state

All features

Retention

Delete after

×

Close

Manage policy

Despues de crear el policy y repositorio se van a crear backups automáticamente cada cierto tiempo dependiendo de lo definido al crear el policy.

16 / 56

Snapshot

Repositories

Policies

Restore Status

Q Search...

Repository

Reload

Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration	Actions
☐ snapshot-elastic-2023.10.20-ppt2fdq7sheb11fiwdpdwg	BackupsElastic	3	11	0	Oct 19, 2023 8:29 PM CST	80s	📄 🗑

Rows per page: 20

< 1 >

Al revisar el repo tambien se podra observar que hay un snapshot disponible

Snapshot and Restore

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

Snapshot

Repositories

Policies

Restore Status

Q Search...

☐ Name ↑

Type

☐ [BackupsElastic](#)

AWS S3

Rows per page: 20

BackupsElastic

Repository type

AWS S3

Repository docs

Snapshots

1 snapshot found

Settings

Bucket

tec-ic4302-02-2023

Client

default

Base path

2021022613/elastic

Verification status

Verify repository

Repository cleanup

You can clean up a repository to delete any unreferenced data from a snapshot. This may provide storage space savings. Note: If you regularly delete snapshots, this functionality will likely not be as beneficial and

Verificar si se subieron al AWS Bucket los backups

```

bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2021022613/elastic/
PRE indices/
PRE tests-029ZYCoAQPWCAGb96iPqQ/
PRE tests-0Dr7orOPQquwAw9-vVhNCg/
PRE tests-9rTJHyn5ScGhEs9GKhmdUw/
PRE tests-0BI95eoTRbGjt7CAZ10x3Q/
PRE tests-aJYsqkfhsZmcvEFJM1FyEg/
PRE tests-pVeX0HAYS3e0GFW1zVWg2Q/
2023-10-20 02:30:05      3332 index-0
2023-10-20 03:07:38       154 index-1
2023-10-20 03:28:53     3609 index-2
2023-10-20 04:28:48     4885 index-3
2023-10-20 05:28:46     6161 index-4
2023-10-20 05:28:47         8 index.latest
2023-10-20 04:28:47    34548 meta-B80vykD1RKYTBDrYWPafdw.dat
2023-10-20 05:28:46    34549 meta-FIwkKg7DQF2rsgp57j3Z3Q.dat
2023-10-20 03:28:52    34544 meta-WEqxs1mpT0ia7dXyCW3EtA.dat
2023-10-20 02:30:04    34313 meta-YPd0QeQGTWSzKvUJU5JaKg.dat
2023-10-20 04:28:47       738 snap-B80vykD1RKYTBDrYWPafdw.dat
2023-10-20 05:28:46       735 snap-FIwkKg7DQF2rsgp57j3Z3Q.dat
2023-10-20 03:28:53       738 snap-WEqxs1mpT0ia7dXyCW3EtA.dat
2023-10-20 02:30:03       709 snap-YPd0QeQGTWSzKvUJU5JaKg.dat
bash-4.2#

```

Restore

Para probar la restauración de datos usando Kibana, se deben haber hecho los pasos anteriores y al menos se debe tener:

- Un índice con datos
- Un Snapshot del Índice

Una vez se tienen esto, la forma más sencilla de probar es desde la sección DEV TOOLS de la interfaz de kibana, borrar el índice y tratar de restaurar sus datos usando el snapshot.

Si hacemos un GET del índice queda claro que si existe y sus datos son los siguientes:

```

GET /indice-backup/_search
{
  "query": {
    "match": {
      "titulo": "Prueba Backups"
    }
  }
}

1 {
2   "took": 1,
3   "timed_out": false,
4   "shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 1,
13      "relation": "eq"
14    },
15    "max_score": 0.5753642,
16    "hits": [
17      {
18        "_index": "indice-backup",
19        "_id": "1LF6TosBN5xh8Gc3X0gH",
20        "_score": 0.5753642,
21        "_source": {
22          "titulo": "Prueba Backups",
23          "contenido": "Este documento es una prueba de backups",
24          "integrantes": [
25            "Granados Retana Diego",
26            "Granados Retana Daniel",
27            "Mora Montes Diego",
28            "Gutierrez Conejo Eduardo",
29            "Cardona Quesada Jose Ricardo"
30          ]
31        }
32      }
33    ]
34  }
35 }

```

Ahora se procede a verificar que si existe un snapshot con ese index:

```

1 GET /indice-backup/_search
2 {
3   "query": {
4     "match": {
5       "titulo": "Prueba Backups"
6     }
7   }
8 }
9
10 GET /_snapshot/BackupsElastic/_all

```

```

1 {
2   "snapshots": [
3     {
4       "snapshot": "indice-backup_20231020",
5       "uuid": "1jst37f1RPeqvsVBQeJ1ew",
6       "repository": "BackupsElastic",
7       "version_id": 8060199,
8       "version": "8.6.1",
9       "indices": [
10        "indice-backup"
11      ],
12      "data_streams": [],
13      "include_global_state": false,
14      "state": "SUCCESS",
15      "start_time": "2023-10-20T23:33:05.740Z",
16      "start_time_in_millis": 1697844785740,
17      "end_time": "2023-10-20T23:33:08.342Z",
18      "end_time_in_millis": 1697844788342,
19      "duration_in_millis": 2602,
20      "failures": [],
21      "shards": {
22        "total": 1,
23        "failed": 0,
24        "successful": 1
25      },
26      "feature_states": []
27    },
28  ],
29  "total": 1,
30  "remaining": 0
31 }

```

Para esto se uso el comando: **GET /_snapshot/BackupsElastic/_all**

Una vez verificados ambos, se va a borrar el índice de la base de datos elasticSearch:

```

1 GET /indice-backup/_search
2 {
3   "query": {
4     "match": {
5       "titulo": "Prueba Backups"
6     }
7   }
8 }
9
10 GET /_snapshot/BackupsElastic/_all
11
12 DELETE /indice-backup

```

```

1 {
2   "acknowledged": true
3 }

```

Se utiliza el comando: **DELETE /indice-backup**

Revisando si todavía existen los datos usando GET, se obtiene:

```

1 GET /indice-backup/_search
2 {
3   "query": {
4     "match": {
5       "titulo": "Prueba Backups"
6     }
7   }
8 }
9
10 GET /_snapshot/BackupsElastic/_all

```

```

1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [indice-backup]",
7         "resource.type": "index_or_alias",
8         "resource.id": "indice-backup",
9         "index_uuid": "_na_",
10        "index": "indice-backup"
11      }
12    ],

```

Restauración

Una vez eliminado, hay dos maneras de restaurar el índice dentro de Kibana:

Comandos

La primera opción para restaurar el índice o la base es utilizando comandos y el nombre del snapshot que se quiere restaurar, el comando sería similar a esto:

POST /_snapshot/[Nombre Repo]/[Nombre Snapshot]/_restore { "indices": "[Indice a restaurar]", "include_global_state": false }

En el caso de nuestro ejemplo se ejecuta el siguiente comando:

POST /_snapshot/BackupsElastic/indice-backup_20231020/_restore { "indices": "indice-backup", "include_global_state": false }

```
1 GET /indice-backup/_search
2 {
3   "query": {
4     "match": {
5       "titulo": "Prueba Backups"
6     }
7   }
8 }
9
10 GET /_snapshot/BackupsElastic/_all
11
12 DELETE /indice-backup
13
14 POST /_snapshot/BackupsElastic/indice-backup_20231020/_restore
15 {
16   "indices": "indice-backup",
17   "include_global_state": false
18 }
```

```
1 {
2   "accepted": true
3 }
```

Ahora si probamos el GET, deberían de volver a aparecer los datos del Index:

```
1 GET /indice-backup/_search
2 {
3   "query": {
4     "match": {
5       "titulo": "Prueba Backups"
6     }
7   }
8 }
9
10 GET /_snapshot/BackupsElastic/_all
11
12 DELETE /indice-backup
13
14 POST /_snapshot/BackupsElastic/indice-backup_20231020/_restore
15 {
16   "indices": "indice-backup",
17   "include_global_state": false
18 }
```

```
1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 1,
13      "relation": "eq"
14    },
15    "max_score": 0.5753642,
16    "hits": [
17      {
18        "_index": "indice-backup",
19        "_id": "ILF6TosBW5xh8Gc3X0gH",
20        "_score": 0.5753642,
21        "_source": {
22          "titulo": "Prueba Backups",
23          "contenido": "Este documento es una prueba de backups",
24          "integrantes": [
25            "Granados Retana Diego",
26            "Granados Retana Daniel",
27            "Mora Montes Diego",
28            "Gutierrez Conejo Eduardo",
29            "Cardona Quesada Jose Ricardo"
30          ]
31        }
32      }
33    ]
34  }
35 }
```

Usando Interfaz Snapshot and Restore

La segunda opción para restaurar un índice eliminado es a través de la Sección Snapshot and Restore dentro de la Interfaz de Kibana en la sección de Stack Management. Una vez aqui buscamos lo siguiente:

☐ Snapshot	Repository	Indices	Shards	Failed shards	Date created ↓	Duration	Actions
☐ snapshot-prueba-elastic-za4l15cesian4z5ezsmkrg	BackupsElastic	1	9	0	Oct 22, 2023 6:59 PM CST	4s	
☐ snapshot-prueba-elastic-rjkrj6ds2-c1oyq5qcwva	BackupsElastic	1	9	0	Oct 22, 2023 5:59 PM CST	4s	
☐ snapshot-prueba-elastic-m4thordbri2qdkaxv2n8xg	BackupsElastic	1	9	0	Oct 22, 2023 4:59 PM CST	4s	
☐ snapshot-prueba-elastic-qzhcuudwrrsd8aodqnbxw	BackupsElastic	1	9	0	Oct 22, 2023 3:59 PM CST	4s	
☐ snapshot-prueba-elastic-hfkgvti-s56khs1jhjts6w	BackupsElastic	1	9	0	Oct 22, 2023 2:59 PM CST	5s	
☐ snapshot-prueba-elastic-lpx9nubhtwskvmuzyn5bfa	BackupsElastic	1	9	0	Oct 22, 2023 1:59 PM CST	4s	
☐ snapshot-prueba-elastic-22r2t5-jsolpg4eoo4b3nw	BackupsElastic	1	9	0	Oct 22, 2023 12:59 PM CST	5s	
☐ snapshot-prueba-elastic-taegbq4sstwgnoc6_i0ag	BackupsElastic	1	9	0	Oct 22, 2023 11:59 AM CST	5s	
☐ snapshot-prueba-elastic-	BackupsElastic	1	9	0	Oct 22, 2023 11:08 AM CST	5s	

Aqui aparecerán todos los snapshots creados, ya sea que se crearon manualmente con comandos o a través de una política. Estos se encuentran en orden descendente del creado más reciente al más antiguo. Se recomienda usar snapshots más nuevos para regresar a un estado más reciente de la base de datos en caso de un fallo.

Se selecciona el snapshot que se quiera restaurar y se clickea para ver detalles de este. Una vez abierto podremos ver la opción de restaurar:

SnapshotsRepositoriesPoliciesRestore Status

Q Search...

Snapshot	Repository	Indices	Shards	Failed shards
☐ snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-rjkrj6ds2-c1oyq5qcvwa	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-m4thordbri2qdkaxv2n8xg	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-qzhcuudwrresd8aodqnbxw	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-hfkgvti-s56khs1jhjts6w	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-lpx9nubhtwskvmuzyn5bfa	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-22r2t5-jsoipg4eoo4b3nw	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-taegbq4sstwwgnoc6_ioag	BackupsElastic	1	9	0
☐ snapshot-prueba-elastic-telxvmskfuw4mmsmshd4	BackupsElastic	1	9	0

snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg

'BackupsElastic' repository

SummaryFailed indices (0)

Version / Version ID8.6.1 / 8060199

State✓ Snapshot complete

Includes global stateYes

Indices (1)index-backup

Data streams (0)

Start timeOct 22, 2023 6:59 PM CST

Created bySnapshotsPrueba-Elastic

UUIDfS7JLuSEQiuHywMIqZZOCA

Duration4 seconds

Include feature state fromgeolipkibanasecurity

End timeOct 22, 2023 7:00 PM CST

Close

Delete

Restore

Si seleccionamos esta opción se desplegará un menú interactivo de restauración donde se pueden seleccionar los índices del snapshot que se quieren restaurar, si se quieren mantener los mismos nombres para los índices o cambiarlos, etc. A continuación se presenta este menú:

Restore 'snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg'

1Logistics2Index settings3Review

Restore details

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

☒ All data streams and indices

Rename data streams and indices

Renames data streams and indices on restore. Ensure that a matching index template exists for renamed data streams.

☐ Rename data streams and indices

Partial restore

Allows restore of indices that don't have snapshots of all shards.

☐ Partial restore

Restore global state

Restores the global cluster state as part of the snapshot.

☐ Restore global state

Restore feature state

Restores the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security.

☐ Restore feature state from

Restore aliases

Restores index aliases along with their associated indices.

☒ Restore aliases

> Next

Snapshot and Restore docs

Seleccionamos las configuraciones que se deseen aplicar al restore y se pasa a la siguiente página:

Restore 'snapshot-prueba-elastic-za4l15ceslan4z5ezsmkrg'



Review restore details

Summary JSON

Logistics

Data streams and indices

All indices

Include feature state

No

Index settings

No index setting modifications

< Back

✓ Restore snapshot

Se le da restore snapshot y listo, los datos de la base junto con los índices y configuraciones en el snapshot se restauraron a la base.

Pruebas Realizadas

Para las pruebas realizadas en elasticSearch, se crearon dos índices:

```
1 PUT pokemon-names
2 {
3   "mappings": {
4     "properties": {
5       "name": { "type": "keyword" }
6     }
7   }
8 }
9
10
11 PUT pokemon-regions
12 {
13   "mappings": {
14     "properties": {
15       "region": { "type": "keyword" }
16     }
17   }
18 }
```

A continuación se presentan los comandos para crear los índices e insertar datos en cada uno:

pokemon-names

```
PUT pokemon-names { "mappings": { "properties": { "name": { "type": "keyword" } } } }
```

>>>> Datos:

```
POST pokemon-names/_doc { "name": "Pikachu" }
```

```
POST pokemon-names/_doc
{ "name": "Charmander" }
```

Consultando al índice se obtienen los datos:

```

1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-names",
19        "_id": "4GZcWosB2vwyjbG-KKCs",
20        "_score": 1,
21        "_source": {
22          "name": "Pikachu"
23        }
24      },
25      {
26        "_index": "pokemon-names",
27        "_id": "4WZcWosB2vwyjbG-MqAX",
28        "_score": 1,
29        "_source": {
30          "name": "Charmander"
31        }
32      }
33    ]
34  }
35 }

```

El comando para consultar el índice es: **GET pokemon-names/_search**

pokemon-regions

PUT pokemon-regions { "mappings": { "properties": { "region": { "type": "keyword" } } } }

>>>> Datos:

POST pokemon-regions/_doc { "region": "Kanto" }

POST pokemon-regions/_doc { "region": "Johto" }

Consultando al indice se obtienen los datos:

```
1 {  
2   "took": 1,  
3   "timed_out": false,  
4   "_shards": {  
5     "total": 1,  
6     "successful": 1,  
7     "skipped": 0,  
8     "failed": 0  
9   },  
10  "hits": {  
11    "total": {  
12      "value": 2,  
13      "relation": "eq"  
14    },  
15    "max_score": 1,  
16    "hits": [  
17      {  
18        "_index": "pokemon-regions",  
19        "_id": "4mZcWosB2vwyjbG-gqCm",  
20        "_score": 1,  
21        "_source": {  
22          "region": "Kanto"  
23        }  
24      },  
25      {  
26        "_index": "pokemon-regions",  
27        "_id": "42ZcWosB2vwyjbG-iqCD",  
28        "_score": 1,  
29        "_source": {  
30          "region": "Johto"  
31        }  
32      }  
33    ]  
34  }  
35 }
```

En base a estos índices se estableció una política que guarde los datos asociados a estos dos índices cada hora. La política es:



snapshots-pokemon

Summary

History

Snapshots taken	Failures	Deleted	Deletion failures
0	0	0	0

General

Version	Last modified
1	Oct 22, 2023 8:47 PM CST
Snapshot name	Repository
pokemon-snapshots-	BackupsElastic
Schedule	Next snapshot
0 55 * * * ?	Oct 22, 2023 8:55 PM CST
Data streams and indices	Ignore unavailable indices
pokemon-names pokemon-regions	No
Include global state	Include feature state
Yes	All features
Allow partial shards	
No	

Ahora se van a borrar los dos índices y tratar de restaurarlos a través de hacer 2 experimentos:

Se borran los índices:

DELETE pokemon-names

DELETE pokemon-regions

Probando obtener los datos:

```
1
2
3
4
5
6
7 DELETE pokemon-names
8
9 DELETE pokemon-regions
10
11 GET pokemon-regions/_search
12
13 GET pokemon-names/_search
14
15
16
17
18
19
20
21
22
```

```
1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-regions]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-regions",
9         "index_uuid": "na",
10        "index": "pokemon-regions"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-regions]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-regions",
17    "index_uuid": "na",
18    "index": "pokemon-regions"
19  },
20  "status": 404
21 }
```

Experimento #1 (Restaurar Un solo índice y el otro no)

Restore 'pokemon-snapshots--fdbesic8rcuu8h1nedh0tw'

1

Logistics

2

Index settings

3

Review

Restore details

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

☐ All data streams and indices

Select data streams and indices

Use index patterns

Filter options

pokemon-names

✓

pokemon-regions

↩

1 index and 0 data streams will be restored. [Deselect all](#)

☒ Rename data streams and indices

Capture pattern

Replacement pattern

pokemon-regions

pokemon-regions12

Use regular expressions

Se va a restaurar solamente el índice **pokemon-region** con el nombre "**pokemon-regions12**"

Haciendo el restore, se consultan ambos índices:

Consultando el índice **pokemon-regions12**

26 / 56

```

1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-regions12",
19        "_id": "4mZcWosB2vwyjbG-gqCm",
20        "_score": 1,
21        "_source": {
22          "region": "Kanto"
23        }
24      },
25      {
26        "_index": "pokemon-regions12",
27        "_id": "42ZcWosB2vwyjbG-iqCD",
28        "_score": 1,
29        "_source": {
30          "region": "Johto"
31        }
32      }
33    ]
34  }
35 }

```

Consultando el índice pokemon-regions

```

1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-regions]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-regions",
9         "index_uuid": "_na_",
10        "index": "pokemon-regions"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-regions]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-regions",
17    "index_uuid": "_na_",
18    "index": "pokemon-regions"
19  },
20  "status": 404
21 }

```

Como se puede ver ahora se restauró el índice pokemon-regions con el nombre **pokemon-regions12**.

También se puede observar que al consultar el índice **pokemon-names**, ya no existe:

```
1 {
2   "error": {
3     "root_cause": [
4       {
5         "type": "index_not_found_exception",
6         "reason": "no such index [pokemon-names]",
7         "resource.type": "index_or_alias",
8         "resource.id": "pokemon-names",
9         "index_uuid": "_na_",
10        "index": "pokemon-names"
11      }
12    ],
13    "type": "index_not_found_exception",
14    "reason": "no such index [pokemon-names]",
15    "resource.type": "index_or_alias",
16    "resource.id": "pokemon-names",
17    "index_uuid": "_na_",
18    "index": "pokemon-names"
19  },
20  "status": 404
21 }
```

Experimento #2 (Restaurar ambos índices después de Experimento1)

Para este experimento se hace un restore normal del snapshot completo:

Restore 'pokemon-snapshots--fdbesic8rcuu8h1nedh0tw'

1

2

3

Logistics

Index settings

Review

Restore details

[Snapshot and Restore docs](#)

Data streams and indices

Creates new data streams and indices if they don't exist. Opens existing indices, including backing indices for a data stream, if they are closed and have the same number of shards as the snapshot index.

☒ All data streams and indices

Rename data streams and indices

Renames data streams and indices on restore. Ensure that a matching index template exists for renamed data streams.

☐ Rename data streams and indices

Partial restore

Allows restore of indices that don't have snapshots of all shards.

☐ Partial restore

Restore global state

Restores the global cluster state as part of the snapshot.

☐ Restore global state

Restore feature state

Restores the configuration, history, and other data stored in Elasticsearch by a feature such as Elasticsearch security.

☐ Restore feature state from

Restore aliases

Restores index aliases along with their associated indices.

☒ Restore aliases

> Next

Después de hacer esto, se puede observar en la sección de Restored dentro de la interfaz de Kibana lo siguiente:

28 / 56

Snapshot and Restore

[Snapshot and Restore docs](#)

Use repositories to store and recover backups of your Elasticsearch indices and clusters.

SnapshotsRepositoriesPoliciesRestore Status

Refresh data every 30 seconds

Index	Status ↑	Last activity	Shards completed	Shards in progress
pokemon-regions	Complete	Oct 22, 2023 9:47 PM CST	1	0
pokemon-names	Complete	Oct 22, 2023 9:47 PM CST	1	0
pokemon-regions12	Complete	Oct 22, 2023 9:32 PM CST	1	0

Rows per page: 20

< 1 >

Se muestran los archivos de los 3 índices restaurados, debido a que en el experimento 1 no se restauró el índice **Pokemon-regions** con el mismo nombre, se restauran 2 veces en vez de sobrescribir el índice restaurado previamente con la versión nueva del snapshot

```
1 {
2   "took": 1,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-regions",
19        "_id": "4mZcWosB2vwyjbG-gqCm",
20        "_score": 1,
21        "_source": {
22          "region": "Kanto"
23        }
24      },
25      {
26        "_index": "pokemon-regions",
27        "_id": "42ZcWosB2vwyjbG-iqCD",
28        "_score": 1,
29        "_source": {
30          "region": "Johto"
31        }
32      }
33    ]
34  }
35 }
```

Revisando **Pokemon-regions**

Pokemon-regions12:

```
1 {  
2   "took": 1,  
3   "timed_out": false,  
4   "_shards": {  
5     "total": 1,  
6     "successful": 1,  
7     "skipped": 0,  
8     "failed": 0  
9   },  
10  "hits": {  
11    "total": {  
12      "value": 2,  
13      "relation": "eq"  
14    },  
15    "max_score": 1,  
16    "hits": [  
17      {  
18        "_index": "pokemon-regions12",  
19        "_id": "4mZcWosB2vwyjbG-gqCm",  
20        "_score": 1,  
21        "_source": {  
22          "region": "Kanto"  
23        }  
24      },  
25      {  
26        "_index": "pokemon-regions12",  
27        "_id": "42ZcWosB2vwyjbG-iqCD",  
28        "_score": 1,  
29        "_source": {  
30          "region": "Johto"  
31        }  
32      }  
33    ]  
34  }  
35 }
```

Y si se trata de obtener **Pokemon-names**

```

1 {
2   "took": 0,
3   "timed_out": false,
4   "_shards": {
5     "total": 1,
6     "successful": 1,
7     "skipped": 0,
8     "failed": 0
9   },
10  "hits": {
11    "total": {
12      "value": 2,
13      "relation": "eq"
14    },
15    "max_score": 1,
16    "hits": [
17      {
18        "_index": "pokemon-names",
19        "_id": "4GZcWosB2vwyjbG-KKCs",
20        "_score": 1,
21        "_source": {
22          "name": "Pikachu"
23        }
24      },
25      {
26        "_index": "pokemon-names",
27        "_id": "4WZcWosB2vwyjbG-MqAX",
28        "_score": 1,
29        "_source": {
30          "name": "Charmander"
31        }
32      }
33    ]
34  }
35 }

```

Como se puede ver, cuando se restaura un índice desde un snapshot, se está creando un índice nuevo a partir de una copia de seguridad. Si se restaura un índice con el mismo nombre que ya existía previamente, Elasticsearch sobrescribirá el índice existente con la versión restaurada desde el snapshot.

Sin embargo, si se restaura el índice con un nombre diferente al que tenía originalmente, Elasticsearch creará un nuevo índice con ese nuevo nombre, en lugar de sobrescribir.

PostgreSQL

Para realizar la instalación de PostgreSQL, se utilizó la versión del [helm chart preparada por Bitnami](#). Se utilizó la versión de la [imagen 14.9.0](#) debido a que la versión que está incluida en el [repositorio de extras de Amazon Linux es PostgreSQL 14](#). Para realizar la instalación, se habilita en el `databases/values.yaml` y en `backups/values.yaml` en el folder de `helm`. En el archivo de `databases/values.yaml`, se especifica la base de datos que se crea en el servidor y adicionalmente el usuario y contraseña que se va a crear. El contenedor está configurado para correr con 1 CPU y 4GB de RAM.

```

15 postgresql:
16   enabled: false
17   image:
18     tag: 14.9.0
19   primary:
20     resources:
21       limits:
22         cpu: "1"
23         memory: "4Gi"
24   auth:
25     database: "Pokemon"
26     username: "admin"
27     password: "admin"

```

Si se desea realizar un backup, en el archivo de backups/values.yaml se pone el campo de `_postgresql.config.type_`. Adicionalmente, se especifica la base de datos que se desea respaldar y un usuario registrado con su contraseña que es el que ingresará a realizar el backup. Los backups de PostgreSQL se van a guardar en `s3://tec-ic4302-02-2023/2019073558/postgresql/`. El campo de `schedule` debe contener un horario válido para un [CronJob](#). En nuestra configuración, está configurado para correr dos veces al día, 1 vez a medio día y otra vez a media noche.

```

31 postgresql:
32   enabled: false
33   config:
34     namespace: default
35     host: databases-postgresql.default.svc.cluster.local
36     db: "Pokemon"
37     user: "admin"
38     password: "admin"
39     port: "5432"
40     bucketName: tec-ic4302-02-2023
41     path: 2019073558/postgresql
42     maxBackups: 3
43     name: Pokemon-202310211634.sql
44     schedule: "0 */12 * * *"
45     diskSize: 2
46     storageClass: hostpath
47     provider: aws
48     type: backup

```

Luego se puede correr el script de `install.sh` en helm para correr el backup. Cuando se instala en modo "backup", se crean dos objetos principales de Kubernetes: un CronJob que se va a ejecutar dos veces al día cada 12 horas, y un Job que se ejecuta de una vez y realiza el respaldo a la base de datos. Estos jobs van a ejecutar el script `TC2/helm/backups/scripts/postgresqlBackup.sh`. Este script utiliza la herramienta de `pg_dump` para realizar el backup. Se usan las opciones de `-h`, `-U` y `-p` para señalar la información de conexión a la base de datos. `-h` indica el host, el cual es el nombre del servicio de la base de datos de PostgreSQL. En el values.yaml del helm chart, se especifica como el FQDN de este servicio. El `-U` representa el usuario, el cual es `admin` en la configuración. No se especifica la contraseña directamente en el comando. Sin embargo, este usa una variable de entorno que se llama `PGPASSWORD` donde la almacena. Por lo tanto, en el template de

`backup.yaml` del helm chart, se especifica la variable de entorno `PGPASSWORD` y se recibe como el campo de `postgresql.config.password`. Aquí se establece la contraseña para el uso. Finalmente, `-p` se usa para indicar el puerto de la base de datos, que en PostgreSQL usualmente es 5432. Luego, las otras opciones de `-w` es para que no solicite un prompt para ingresar la contraseña. `-c` es para indicar que se incluyan instrucciones de "DROP" para la creación de objetos. Esto vuelve a crear todos los objetos dentro de la base de datos. Sin embargo, la base de datos debe existir en el servidor de la base de datos. Finalmente, se utiliza la siguiente instrucción para subir el backup al bucket S3 de aws: `aws s3 cp /pgdump/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/ --recursive` Este script crea backups con el nombre `DBNAME-DATE.sql`

```
helm > backups > scripts > $ postgresqlBackup.sh
1  #!/bin/bash
2  # https://devopscube.com/install-configure-postgresql-amazon-linux/
3  DATE=$(date +%Y%m%d%H%M)
4
5  mkdir -p /pgdump/$DATE
6  yum update -y
7
8  amazon-linux-extras enable postgresql14
9  yum clean metadata
10 yum install postgresql -y
11
12 pg_dump -h $POSTGRES_HOST -U $POSTGRES_USER -p $POSTGRES_PORT -w -c $POSTGRES_DB > /pgdump/$DATE/$POSTGRES_DB-$DATE.sql
13 echo "Backed up"
14 aws s3 cp /pgdump/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/ --recursive
15 aws s3 ls s3://$BUCKET_NAME/$BACKUP_PATH/
16 rm -rf /pgdump/$DATE
17 echo "Finished!"
18
```

El restore de PostgreSQL funciona muy similarmente. Para activarlo, hay que cambiar el type en el `values.yaml` a `restore` y especificar el nombre del backup que queremos restaurar en el campo de `name`:

```
31 postgresql:
32   enabled: true
33   config:
34     namespace: default
35     host: databases-postgresql.default.svc.cluster.local
36     db: "Pokemon"
37     user: "admin"
38     password: "admin"
39     port: "5432"
40     bucketName: tec-ic4302-02-2023
41     path: 2019073558/postgresql
42     maxBackups: 3
43     name: Pokemon-202310212135.sql
44     schedule: "0 */12 * * *"
45     diskSize: 2
46     storageClass: hostpath
47     provider: aws
48     type: restore
```

El script que ejecuta el restore utiliza la herramienta de `psql`, el cual es una terminal interactiva para Postgres. En el script, la configuración de opciones es muy igual a la del script de backups. Se conecta con el host, el usuario, y el puerto y restaura la base de datos dada al final. El backup lo recibe como entrada. Este backup es descargado del bucket de AWS a partir del path y el name dado en la configuración en el `values.yaml`.

```

1  #!/bin/bash
2  # https://devopscube.com/install-configure-postgresql-amazon-linux/
3  DIRECTORY="/pgdump"
4  mkdir -p $DIRECTORY || echo "Failed to create directory"
5
6  yum update -y
7
8  amazon-linux-extras enable postgresql14
9  yum clean metadata
10 yum install postgresql -y
11
12 aws s3 cp s3://$BUCKET_NAME/$BACKUP_PATH/$RESTORE_FILE $DIRECTORY
13 echo "Downloaded"
14
15 psql -h $POSTGRES_HOST -U $POSTGRES_USER -p $POSTGRES_PORT -w $POSTGRES_DB < $DIRECTORY/$RESTORE_FILE
16 echo "Restored"
17 rm -rf $DIRECTORY/$RESTORE_FILE
18 echo "Finished!"

```

Ejemplo de Backup y Restore:

Para tener más control, podemos instalar los helm charts por separado. Primero, hay que instalar bootstrap. Ubicándonos en el folder de helm, hacemos: `helm install bootstrap bootstrap` Luego, tras asegurarnos de que PostgreSQL está habilitado en el values.yaml de databases, hacemos: `helm install databases databases` Esto crea el pod de la base de datos.

Pods											
2 Items			Namespace: default		Search Pods...						
☐ Name	Namespace	Contai...	CPU	Memory	Restarts	Controlled By	Node	QoS	Age	Status	
☐ debug-pod	default	■	N/A	N/A	0		docker-de	BestEffort	4h44m	Running	
☐ databases-postgresql-0	default	■	N/A	N/A	0	StatefulSet	docker-de	Burstable	4h52m	Running	

Para insertar registros en la base de datos, podemos hacer un portforward del servicio de la base de datos. Para esto, en Lens, vamos a Network, Services, databases-postgresql, lo abrimos y bajamos hasta que aparezca el puerto. Aquí podemos abrir el portforward. Esto nos va a abrir una ventana del navegador con el puerto que se está exponiendo en la máquina local.

DD

docker-desktop

Controllers

Jobs

Cron Jobs

Config

Config Maps

Secrets

Resource Quotas

Limit Ranges

Horizontal Pod Autoscalers

Vertical Pod Autoscalers

Pod Disruption Budgets

Priority Classes

Runtime Classes

Leases

Mutating Webhook Configs

Validating Webhook Configs

Network

Services

Endpoints

Ingresses

Ingress Classes

Network Policies

Port Forwarding

Storage

Persistent Volume Claims

Persistent Volumes

Storage Classes

Services

Endpoints

Ingresses

Ingress Classes

Network Policies

Port Forwarding

Storage

Persistent Volume Claims

Persistent Volumes

Storage Classes

3 Items

☐ Name	Namespace	Type	Cluster IP	Ports
☐ databases-postgresql	default	ClusterIP	10.105.208.47	5432:tcp-p
☐ databases-postgresql-hl	default	ClusterIP	None	5432:tcp-p
☐ kubernetes	default	ClusterIP	10.96.0.1	443:6443/TCP

Job postgresql-restore-0

+

Namespace = Owner Job postgresql-restore-0 Pod All Pods

Container All containers

1

(1 row)

ALTER TABLE

REVOKE

REVOKE

GRANT

GRANT

Restored

Finished!

download: s3://tec-1c4302-02-2023/2019073558/postgresql/Pokemon-202310211634.sql to .../pgdump/Pokemon-202310211634.sql

Logs from 10/21/2023, 10:56:38 AM

Service: databases-postgresql

app.kubernetes.io/name=postgresql

helm.sh/chart=postgresql-12.10.0

Annotations

meta.helm.sh/release-name=databases

meta.helm.sh/release-namespace=default

Selector

app.kubernetes.io/component=primary

app.kubernetes.io/instance=databases

app.kubernetes.io/name=postgresql

Type

ClusterIP

Session Affinity

None

Connection

Cluster IP

10.105.208.47

Cluster IPs

10.105.208.47

IP families

IPv4

IP family policy

SingleStack

Ports

5432:tcp-postgresql/TCP

Stop/Remove

Endpoint

Name

Endpoints

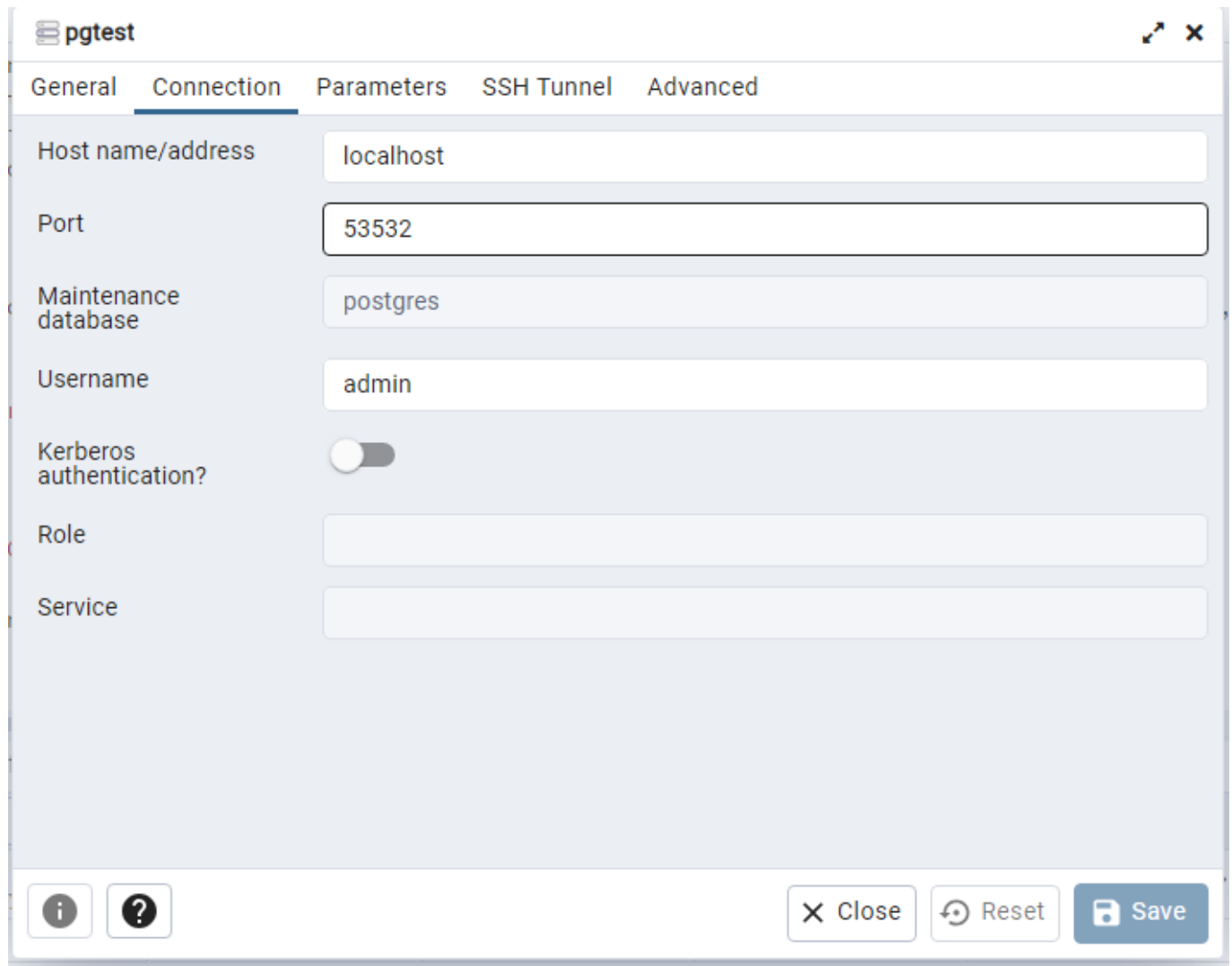
databases-postgresql

10.1.0.6:5432

Events

No events found

Luego, podemos abrir una herramienta de Postgres como [pgAdmin](#) y conectarnos al servidor de PostgreSQL en el puerto abierto por el portforward. Ahí, al registrar el servidor, podemos ponerle un nombre y podemos ingresar con el usuario definido en el helm chart.



The screenshot shows the 'pgtest' application window with the 'Connection' tab selected. The interface includes a top navigation bar with tabs: 'General', 'Connection', 'Parameters', 'SSH Tunnel', and 'Advanced'. The 'Connection' tab contains the following fields and controls:

- Host name/address:** A text input field containing 'localhost'.
- Port:** A text input field containing '53532'.
- Maintenance database:** A text input field containing 'postgres'.
- Username:** A text input field containing 'admin'.
- Kerberos authentication?:** A toggle switch that is currently turned off.
- Role:** An empty text input field.
- Service:** An empty text input field.

At the bottom of the window, there are three buttons: 'Close' (with an 'X' icon), 'Reset' (with a circular arrow icon), and 'Save' (with a floppy disk icon). There are also information and help icons on the bottom left.

Aquí podemos correr consultas, como crear una tabla e insertar un registro:

Query

```

1 CREATE TABLE IF NOT EXISTS pokemons (
2     primaryKey SERIAL PRIMARY KEY,
3     pokemonId VARCHAR(255),
4     PokemonName VARCHAR(255),
5     Type1 VARCHAR(255),
6     Type2 VARCHAR(255),
7     Category VARCHAR(255),
8     Heightf VARCHAR(255),
9     Heightm VARCHAR(255),
10    Weightlbs VARCHAR(255),
11    Weightkg VARCHAR(255),
12    CaptureRate VARCHAR(255),
13    EggSteps VARCHAR(255),
14    ExpGroup VARCHAR(255),
15    Total VARCHAR(255),
16    HP VARCHAR(255),
17    Attack VARCHAR(255),
18    Defense VARCHAR(255),
19    SpAttack VARCHAR(255),
20    SpDefense VARCHAR(255),
21    Speed VARCHAR(255)
22 );
23
24 INSERT INTO pokemons (
25     pokemonId, PokemonName, Type1, Type2, Category, Heightf, Heightm, Weightlbs, Weightkg, CaptureRate, EggSteps, ExpGroup, T
26 )
27 VALUES (
28     '001', 'Bulbasaur', 'Grass', 'Poison', 'Seed Pokémon', '2.04', '0.7', '15.2', '6.9', '45', '5120', 'Medium Slow', '318',
29 );
30

```

Data Output

	primaryKey [PK] integer	pokemonid character varying (255)	pokemonname character varying (255)	type1 character varying (255)	type2 character varying (255)	category character varying (255)	heightf character varying (255)	heightm character varying (255)
1	1	001	Bulbasaur	Grass	Poison	Seed Pokémon	2.04	0.7

Ya con información en la base de datos, podemos instalar el helm chart de backups para que el Job realice el respaldo: `helm install backups backups` Podemos ir a Lens y la sección de Jobs. Aquí podemos abrir la consola del pod y ver el progreso del script. Si sale un error de que no encuentra el script, hay que hacer la instrucción en la consola de y reinstalar el helm chart de backups: `dos2unix backups/scripts/postgresqlBackup.sh`

Overview Pods Deployments Daemon Sets Stateful Sets Replica Sets Replication Controllers **Jobs** Cron Jobs

Jobs 1 item

Name	Namespace	Completions	Age	Conditions
postgresql-backup-0	default	1 / 1	72s	Complete

Job postgresql-backup-0

Namespace: default Owner: Job postgresql-backup-0 Pod: All Pods Container: All containers

Total download size: 1.7 M
Installed size: 6.4 M
Downloading packages:

Package	Size	Progress
postgresql-backup-0	1.7 MB/s	00:01

Running transaction check
Running transaction test
Transaction test succeeded

Installing : libpq-14.8-1.amzn2.0.1.x86_64 1/2
Installing : postgresql-14.8-1.amzn2.0.2.x86_64 2/2
Verifying : libpq-14.8-1.amzn2.0.1.x86_64 1/2
Verifying : postgresql-14.8-1.amzn2.0.2.x86_64 2/2

Installed:
postgresql.x86_64 0:14.8-1.amzn2.0.2

Dependency Installed:
libpq.x86_64 0:14.8-1.amzn2.0.1

Completed!
Backed up
Completed 3.2 KiB/3.2 KiB (5.2 KiB/s) with 1 file(s) remaining

Time	Size	File(s)
2023-10-16 01:37:12	0	Pokemon-202310160136.sql
2023-10-16 01:40:59	0	Pokemon-202310160140.sql
2023-10-16 03:12:57	0	Pokemon-202310160311.sql
2023-10-16 04:02:09	3087	Pokemon-202310160400.sql
2023-10-17 20:14:14	3087	Pokemon-202310172011.sql
2023-10-18 14:16:12	3087	Pokemon-202310181455.sql
2023-10-21 16:35:15	3291	Pokemon-202310211634.sql
2023-10-21 21:35:59	3291	Pokemon-202310212135.sql

Finished!
upload: .../pgdump/202310212135/Pokemon-202310212135.sql to s3://tec-ic4302-02-2023/2019073558/postgresql/Pokemon-202310212135.sql

Al final, podemos usar el debug-pod de /utils para consultar si se subió correctamente el archivo con el comando: `aws s3 ls s3://tec-ic4302-02-2023/2019073558/postgresql/`

```
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023/2019073558/postgresql/
2023-10-16 01:37:12      0 Pokemon-202310160136.sql
2023-10-16 01:40:59      0 Pokemon-202310160140.sql
2023-10-16 03:12:57      0 Pokemon-202310160311.sql
2023-10-16 04:02:05    3087 Pokemon-202310160400.sql
2023-10-17 20:34:14    3087 Pokemon-202310172031.sql
2023-10-18 14:56:12    3087 Pokemon-202310181455.sql
2023-10-21 16:35:15    3291 Pokemon-202310211634.sql
2023-10-21 21:35:59    3291 Pokemon-202310212135.sql
bash-4.2#
```

Aquí podemos ver que el backup creado es Pokemon-202310212135.sql. Ahora vamos a restaurar este. Para probarlo, podemos eliminar el registro insertado anteriormente, para ver cómo se restaura:

```
30
31 DELETE FROM pokemons
32 WHERE pokemonId = '001';
33
34 SELECT * FROM pokemons;
35
36
```

	primarykey [PK] integer	pokemonid character varying (255)	pokemonname character varying (255)	type1 character varying (255)	type2 character varying (255)	category character varying (255)	heightf character varying (255)	hei chi
--	----------------------------	--------------------------------------	--	----------------------------------	----------------------------------	-------------------------------------	------------------------------------	------------

Ahora, escribimos el nombre del backup Pokemon-202310212135.sql en el campo de name en el values.yaml de backups y cambiamos el type a restore. Tras hacer esto, podemos reinstalar el helm chart con: `helm uninstall backups helm install backups backups` Cuando está en modo restore, no crea un CronJob, solo ejecuta un Job. Por lo tanto, podemos ver el progreso del Job en la pestaña de Jobs en Lens:

The screenshot shows the 'Jobs' tab in the Lens interface. A single job, 'postgresql-restore-0', is listed with a completion status of '1 / 1' and a condition of 'Complete'. The job's logs are visible below, showing a series of SQL commands including 'SET', 'ALTER TABLE', 'DROP SEQUENCE', 'DROP TABLE', 'CREATE TABLE', 'ALTER TABLE', 'CREATE SEQUENCE', 'ALTER TABLE', 'ALTER SEQUENCE', 'ALTER TABLE', 'COPY', 'SETVAL', and 'RESTORE'. The final line of the log indicates the restoration of the database from the backup file 'Pokemon-202310212135.sql'.

Al final, podemos saber que el Job terminó cuando la columna de Completions salga en 1/1. Ahora, podemos verificar que el restore se realizó en pgAdmin haciendo un select de la tabla:

```

33
34 SELECT * FROM pokemons;
35
36

```

	primarykey [PK] integer	pokemonid character varying (255)	pokemonname character varying (255)	type1 character varying (255)	type2 character varying (255)	category character varying (255)	heightf character varying (255)	hei cha
1	1	001	Bulbasaur	Grass	Poison	Seed Pokémon	2.04	0.7

MongoDB

Para realizar la instalación de MongoDB, se utilizó la versión del [helm chart preparada por Bitnami](#). Para realizar la instalación, se habilita en el `databases/values.yaml` y en `backups/values.yaml` en el folder de `helm` la opciones de `enabled` para realizar la instalación de la base de datos deseada. En el archivo de `databases/values.yaml`, como no se está indicando algún valor además del `enabled`, la instalación se va a realizar de manera default, es decir se va a realizar una instalación con los parámetros predefinidos por el helmchart.

```

mongodb:
  enabled: true

```

Si se desea realizar un backup, en el archivo de `backups/values.yaml` se pone `type: backup` en el campo de `_mongo.config.type`. En este setup, el job que realiza esta toma los datos del secret de mongo para obtener el acceso a la base de datos y realizar el backup. Los backups de Mongo se van a guardar en `s3://tec-ic4302-02-2023/2019073558/mongo/`. El campo de `schedule` debe contener un horario válido para un [CronJob](#). En nuestra configuración, está configurado para correr dos veces al día, 1 vez a medio día y otra vez a media noche.

```

mongo:
  enabled: false
  config:
    namespace: default
    connectionString: databases-mongodb.default.svc.cluster.local:27017
    bucketName: tec-ic4302-02-2023
    path: 2019073558/mongodb
    maxBackups: 3
    secret: databases-mongodb
    name: "202310202250"
    schedule: "0 */12 * * *"
    diskSize: 1
    storageClass: hostpath
    provider: aws
    type: restore

```

Luego se puede correr el script de `install.sh` en helm para correr el backup. Cuando se instala en modo "backup", se crean dos objetos principales de Kubernetes: un CronJob que se va a ejecutar dos veces al día cada 12 horas, y un Job que se ejecuta de una vez y realiza el respaldo a la base de datos. Estos jobs van a

ejecutar el script `TC2/helm/backups/scripts/mongodbBackup.sh`. Este script utiliza la herramienta de `mongodump` para realizar el backup. Se usan las opciones de `--host`, `-U` y `-p` para señalar la información de conexión a la base de datos. `--host` indica el host, el cual es el nombre del servicio de la base de datos de PostgreSQL. En el `values.yaml` del helm chart, se especifica como el FQDN de este servicio. El `-U` representa el usuario, el cual es `root` en la configuración. No se especifica la contraseña directamente en el comando. Sin embargo, este usa una variable de entorno que se llama `MONGO_PASSWORD` donde la almacena. Por lo tanto, en el template de `backup.yaml` del helm chart, se especifica la variable de entorno `MONGO_PASSWORD` y se recibe como el campo de `mongo.config.password`. Aquí se establece la contraseña para el uso. Finalmente, se utiliza la siguiente instrucción para subir el backup al bucket S3 de aws: `aws s3 cp /mongodump/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/$DATE --recursive` Este script crea backups dentro del S3 dentro de una carpeta con la fecha, un archivo llamado `archive` donde se tiene la información del backup realizado. El restore de Mongo funciona muy similarmente. Para activarlo, hay que cambiar el type en el `values.yaml` a `restore` y especificar el nombre del backup que queremos restaurar en el campo de `name`: El script que ejecuta el restore utiliza la herramienta de `mongodumps` igual que el backup, por lo que la instalación es la misma. En el script, la configuración de opciones es muy igual a la del script de backups. Se conecta con el host, el usuario, y restaura la base de datos dada al final. El backup lo recibe como entrada. Este backup es descargado del bucket de AWS a partir del path y el name dado en la configuración en el `values.yaml` en el valor de `name`.

Ejemplo de Backup y Restore:

Para tener más control, podemos instalar los helm charts por separado. Primero, hay que instalar bootstrap. Ubicándonos en el folder de helm, hacemos: `helm install bootstrap bootstrap` Luego, tras asegurarnos de que Mongo está habilitado en el `values.yaml` de `databases`, hacemos: `helm install databases databases` Esto crea el pod de la base de datos. Para insertar registros en la base de datos, podemos hacer una terminal interactiva con el pod con el comando:

```
kubect1 exec -i -t -n default databases-mongodb-59dbc55f8c-2fndw -c mongodb -- sh
-c "clear; (bash || ash || sh)"
```

Dentro de esta terminal interactiva se pueden realizar todas las operaciones para probar crear una colección y agregar un registro a la base de datos y luego hacer el backup mediante el job, y finalmente borrar los registros que hay en la base de datos para después realizar el restore y observar que se recupera la información que ha sido borrada. En el documento dentro de la carpeta de `dbscripts`, dentro de la carpeta de `MongoDB`, se encuentran los scrips tanto de creación como los de borrado para probar esta funcionalidad.

Datos antes del borrado

```
test> use pokemon_db
switched to db pokemon_db
pokemon_db> db.pokemon.find()
[
  {
    _id: ObjectId("6532e6536dfe1a17c2428c33"),
    name: 'Pikachu',
    type: 'Electric',
    level: 25
  }
]
pokemon_db> █
```

Ya con información en la base de datos, podemos instalar el helm chart de backups para que el Job realice el respaldo: `helm install backups backups` Podemos ir a Lens y la sección de Jobs. Aquí podemos abrir la consola del pod y ver el progreso del script. Si sale un error de que no encuentra el script, hay que hacer la instrucción en la consola de y reinstalar el helm chart de backups: `dos2unix backups/scripts/postgresqlBackup.sh` Al final, podemos usar el debug-pod de /utils para consultar si se subió correctamente el archivo con el comando: `aws s3 ls s3://tec-ic4302-02-2023/2019073558/mongodb/` Aquí podemos ver que el backup que se ha generado. Ahora se procede a realizar el borrado de los datos de mongo:

Datos después del borrado

```
pokemon_db> db.pokemon.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
pokemon_db> db.pokemon.find()

pokemon_db> █
```

Para realizar el restore, se escribe el nombre del backup generado, este se puede consultar utilizando el mismo comando de `aws s3 ls s3://tec-ic4302-02-2023/2019073558/mongodb/`, este dato se agrega en el campo de name en el values.yaml de backups y cambiamos el type a restore. Tras hacer esto, podemos reinstalar el helm chart con: `helm uninstall backups helm install backups backups` Cuando está en modo restore, no crea un CronJob, solo ejecuta un Job. Por lo tanto, podemos ver el progreso del Job en la pestaña de Jobs en Lens: Al final, podemos saber que el Job terminó cuando la columna de Completions salga en 1/1. Ahora mediante un find en la terminal interactiva se puede observar que el restore ha sido exitoso.

Datos despues del restore

```
pokemon_db> db.pokemon.find()
[
  {
    _id: ObjectId("6532e6536dfe1a17c2428c33"),
    name: 'Pikachu',
    type: 'Electric',
    level: 25
  }
]
pokemon_db> █
```

Para realizar la instalación de MongoDB, se utilizó la versión del [helm chart preparada por Bitnami](#). Para realizar la instalación, se habilita en el `databases/values.yaml` y en `backups/values.yaml` en el folder de `helm` la opción de `enabled` para realizar la instalación de la base de datos deseada. En el archivo de `databases/values.yaml`, como no se está indicando algún valor además del `enabled`, la instalación se va a realizar de manera default, es decir se va a realizar una instalación con los parámetros predefinidos por el helmchart.

```
moariadb:
  enabled: true
```

Si se desea realizar un backup, en el archivo de `backups/values.yaml` se pone `type: backup` en el campo de `_maria.config.type`. En este setup, el job que realiza esta toma los datos del secret de `mariadb` para obtener el acceso a la base de datos y realizar el backup. Los backups de MariaDB se van a guardar en `s3://tec-ic4302-02-2023/2019073558/mariadb/`. El campo de `schedule` debe contener un horario válido para un [CronJob](#). En nuestra configuración, está configurado para correr dos veces al día, 1 vez a medio día y otra vez a media noche.

```
maria:
  enabled: false
  config:
    namespace: default
    connectionString: databases-mariadb
    bucketName: tec-ic4302-02-2023
    path: 2019073558/mariadb
    maxBackups: 3
    secret: databases-mariadb
    name: pokemon_db-202310210452.sql
    schedule: "0 */12 * * *"
    diskSize: 1
    storageClass: hostpath
    provider: aws
    type: restore
```

Luego se puede correr el script de `install.sh` en helm para correr el backup. Cuando se instala en modo "backup", se crean dos objetos principales de Kubernetes: un CronJob que se va a ejecutar dos veces al día cada 12 horas, y un Job que se ejecuta de una vez y realiza el respaldo a la base de datos. Estos jobs van a ejecutar el script `TC2/helm/backups/scripts/mariadbBackup.sh`. Este script utiliza la herramienta de [mysqldump](#) para realizar el backup. Se usan las opciones de `-h`, `-U` y `-p` para señalar la información de conexión a la base de datos. `-h` indica el host, el cual es el nombre del servicio de la base de datos de PostgreSQL. En el `values.yaml` del helm chart, se especifica como el FQDN de este servicio. El `-U` representa el usuario, el cual es `root` en la configuración. No se especifica la contraseña directamente en el comando. Sin embargo, este usa una variable de entorno que se llama `MARIA_PASSWORD` donde la almacena. Por lo tanto, en el template de `backup.yaml` del helm chart, se especifica la variable de entorno `MARIA_PASSWORD` y se recibe como el campo de `maria.config.password`. Aquí se establece la contraseña para el uso. Finalmente, se utiliza la siguiente instrucción para subir el backup al bucket S3 de aws: `aws s3 cp /mariadbump/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/ --recursive` Este script crea backups dentro del S3 dentro de un

archivo con la fecha de cuando se realizó el backup. El restore de MariaDB funciona muy similar. Para activarlo, hay que cambiar el type en el values.yaml a restore y especificar el nombre del backup que queremos restaurar en el campo de name: El script que ejecuta el restore utiliza la herramienta de mysqldump igual que el backup, por lo que la instalación es la misma. En el script, la configuración de opciones es muy igual a la del script de backups. Se conecta con el host, el usuario, y restaura la base de datos dada al final. El backup lo recibe como entrada. Este backup es descargado del bucket de AWS a partir del path y el name dado en la configuración en el values.yaml en el valor de *name*.

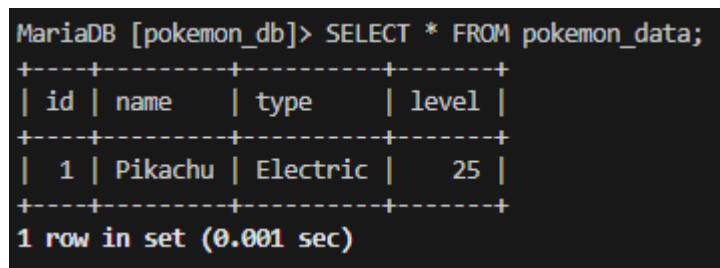
Ejemplo de Backup y Restore:

Para tener más control, podemos instalar los helm charts por separado. Primero, hay que instalar bootstrap. Ubicándonos en el folder de helm, hacemos: `helm install bootstrap bootstrap` Luego, tras asegurarnos de que Mongo está habilitado en el values.yaml de databases, hacemos: `helm install databases databases` Esto crea el pod de la base de datos. Para insertar registros en la base de datos, podemos hacer una terminal interactiva con el pod con el comando:

```
PS C:\Users\joseg> kubectl exec -i -t -n default databases-mariadb-0 -c mariadb --
sh -c "clear; (bash || ash || sh)"
```

Dentro de esta terminal interactiva se pueden realizar todas las operaciones para probar crear una base de datos, tablas y agregar un registro a la base de datos y luego hacer el backup mediante el job, y finalmente borrar los registros que hay en la base de datos para después realizar el restore y observar que se recupera la información que ha sido borrada. En el documento dentro de la carpeta de dbscripts, dentro de la carpeta de MariaDB, se encuentran los scripts tanto de creación como los de borrado para probar esta funcionalidad.

Datos antes del borrado



```
MariaDB [pokemon_db]> SELECT * FROM pokemon_data;
+----+-----+-----+-----+
| id | name  | type   | level |
+----+-----+-----+-----+
|  1 | Pikachu | Electric |    25 |
+----+-----+-----+-----+
1 row in set (0.001 sec)
```

Ya con información en la base de datos, podemos instalar el helm chart de backups para que el Job realice el respaldo: `helm install backups backups` Podemos ir a Lens y la sección de Jobs. Aquí podemos abrir la consola del pod y ver el progreso del script. Si sale un error de que no encuentra el script, hay que hacer la instrucción en la consola de y reinstalar el helm chart de backups: `dos2unix backups/scripts/postgresqlBackup.sh` Al final, podemos usar el debug-pod de /utils para consultar si se subió correctamente el archivo con el comando: `aws s3 ls s3://tec-ic4302-02-2023/2019073558/mariadb/` Aquí podemos ver que el backup que se ha generado. Ahora se procede a realizar el borrado de los datos de MariaDB:

Datos después del borrado

```
MariaDB [pokemon_db]> DELETE FROM pokemon_data;
Query OK, 1 row affected (1.370 sec)

MariaDB [pokemon_db]> SELECT * FROM pokemon_data;
Empty set (0.000 sec)
```

Para realizar el restore, se escribe el nombre del backup generado, este se puede consultar utilizando el mismo comando de `aws s3 ls s3://tec-ic4302-02-2023/2019073558/mariadb/`, este dato se agrega en el campo de name en el values.yaml de backups y cambiamos el type a restore. Tras hacer esto, podemos reinstalar el helm chart con: `helm uninstall backups helm install backups backups` Cuando está en modo restore, no crea un CronJob, solo ejecuta un Job. Por lo tanto, podemos ver el progreso del Job en la pestaña de Jobs en Lens: Al final, podemos saber que el Job terminó cuando la columna de Completions salga en 1/1. Ahora mediante un select * en la terminal interactiva se puede observar que el restore ha sido exitoso.

Datos despues del restore

```
MariaDB [pokemon_db]> SELECT * FROM pokemon_data;
+----+-----+-----+-----+
| id | name  | type   | level |
+----+-----+-----+-----+
| 2  | Pikachu | Electric | 25  |
+----+-----+-----+-----+
1 row in set (0.001 sec)
```

CouchDB

Para realizar la instalacion de CouchDB utilizamos la instalacion del [helm chart hecho por Apache](#). Para realizar la instalación, se habilita en el `databases/values.yaml` y en `backups/values.yaml` en el folder de `helm` la opción de enabled para realizar la instalación de la base de datos deseada. En el archivo de `databases/values.yaml`, se indica la versión de la imagen que se va a utilizar en este caso la 3.3.2, también se define el usuario y la contraseña que se van a utilizar para poder acceder a la base de datos y por último se va a generar un UUID (Universal Unique Identifier) necesario para la creación de la base de datos, este lo creamos aleatoriamente mediante el uso del link (<https://www.uuidgenerator.net/api/version4>).

```
couchdb:
  enabled: true
  clusterSize: 1
  adminUsername: "admin"
  adminPassword: "admin"
  image:
    tag: 3.3.2
  couchdbConfig:
    couchdb:
      uuid: $(shell curl -s https://www.uuidgenerator.net/api/version4 2>/dev/null
| tr -d -)
```

Si se desea realizar un backup, en el archivo de backups/values.yaml se pone `type: backup` en el campo de couchdb.config.type. Los backups de CouchDB se van a guardar en `s3://tec-ic4302-02-`

[2023/2019073558/couchdb/](#). El campo de schedule debe contener un horario válido para un [CronJob](#). En nuestra configuración, está configurado para correr dos veces al día, 1 vez a medio día y otra vez a media noche. El host va a tener la dirección de la base de datos de CouchDB dentro del cluster para que así pueda localizarlo y hacer el backup, la base de datos va a ser la base de datos ya definida de la cual se va a hacer el backup, el port va a ser el predefinido por el helm chart de CouchDB por el cual vamos a tener acceso al API. Y tenemos el user y password que van a ser los mismos que definimos previamente, por ultimo para el caso del restore se le indica un name, que va a ser el nombre del archivo en el cual se está guardando el backup que se quiera restaurar.

```
couchdb:
  enabled: true
  config:
    namespace: default
    host: databases-couchdb.default.svc.cluster.local
    db: "pokemon"
    user: "admin"
    password: "admin"
    port: "5984"
    bucketName: tec-ic4302-02-2023
    path: 2019073558/couchdb
    maxBackups: 3
    name: couchdb-202310230113.json
    schedule: "0 */12 * * *"
    diskSize: 1
    storageClass: hostpath
    provider: aws
    type: backup
```

Luego se puede correr el script de [install.sh](#) en helm para correr el backup. Cuando se instala en modo "backup", se crean dos objetos principales de Kubernetes: un CronJob que se va a ejecutar dos veces al día cada 12 horas, y un Job que se ejecuta de una vez y realiza el respaldo a la base de datos. Estos jobs van a ejecutar el script [TC2/helm/backups/scripts/couchdbB.sh](#). Debido a que CouchDB no tiene definido ningún método para realizar los backups, hacemos uso del [API de CouchDB](#) para así extraer los documentos que contenga la base de datos y poder crear un respaldo de estos. Para realizar el backup utilizamos el siguiente método:

```
URL="http://$COUCHDB_USER:$COUCHDB_PSW@$COUCHDB_HOST:$COUCHDB_PORT/$COUCHDB_DB/_all_docs?include_docs=true"
curl -X GET "$URL" -H "Accept: application/json" | jq '{ docs: [.rows[] | .doc] }'
> /couchdump/$DATE/couchdb-$DATE.json
```

En donde de primero definimos el path que vamos a utilizar para hacer la llamada al API, en este caso utilizamos unas variables de entorno definidas con los datos del values.yaml, el endpoint que utilizamos es **db/_all_docs?include_docs=true**, el cual nos va a retornar todos los documentos de la base de datos que indiquemos para así poder almacenarlos en un archivo .json, ya teniendo el path para el llamado utilizamos el método **curl** para hacer el llamado, en este indicamos que es un GET, y le definimos el url, e indicamos el tipo

de dato que esperamos de vuelta, al mismo tiempo estamos utilizando el método **jq** para así cambiarle el formato al archivo de respuesta para así tenerlo en un formato que luego pueda ser utilizado para el recovery , y por ultimo le indicamos el path en el cual queremos que se almacene y el nombre del archivo que queremos que tenga.

Finalmente, se utiliza la siguiente instrucción para subir el backup al bucket S3 de aws: **aws s3 cp /couchdump/\$DATE s3://\$BUCKET_NAME/\$BACKUP_PATH/ --recursive** Este script crea backups dentro del S3 dentro de un archivo con la fecha de cuando se realizó el backup. En el caso del restore, es bastante similar al backup pero tiene ciertas diferencias, entre estas esta que se utiliza el endpoint del API **_bulk_docs** al cual se le da el archivo .json extraído del path y con el nombre indicado e inserta en la base de datos todos los documentos dentro de este archivo.

Ejemplo de Backup y Restore:

Para tener más control, podemos instalar los helm charts por separado. Primero, hay que instalar bootstrap. Ubicándonos en el folder de helm, hacemos: **helm install bootstrap bootstrap** Luego, tras asegurarnos de que Couchdb está habilitado en el values.yaml de databases, hacemos: **helm install databases databases** Esto crea el pod de la base de datos. Una vez creado hacemos un port foward dentro de Lens del puerto de CouchDB para asi poder hacer uso del API en herramientas como Postman o Thunderclient. Con esto ya podemos hacer la creación de la base de datos y la inserción de los archivos mediante llamados al API, para este ejemplo los llamados utilizados son los siguientes.

```
PUT http://127.0.0.1:50209/pokemon
PUT http://127.0.0.1:50209/pokemon/001
{"name": "Pikachu", "type": "Electric", "level": 25}
```

Ya con información en la base de datos, podemos instalar el helm chart de backups para que el Job realice el respaldo: **helm install backups backups**

Podemos ir a Lens y la sección de Jobs. Aquí podemos abrir la consola del pod y ver el progreso del script. Si sale un error de que no encuentra el script, hay que hacer la instrucción en la consola de y reinstalar el helm chart de backups: **dos2unix backups/scripts/postgresqlBackup.sh** Una vez creado el backup volvemos a Postman y eliminamos nuestros datos en este caso el llamado utilizado es: **DELETE**

http://127.0.0.1:50209/pokemon/001?rev=1-a11edc90c0d2388866469ce2f5235047 Donde el rev es autogenerado por CouchDB por cada registro, podemos ver cuál es el rev de nuestro registro con el llamado:

GET http://127.0.0.1:50209/pokemon/_all_docs?include_docs=true Para realizar el restore, se escribe el nombre del backup generado, este lo recibimos cuando se ejecute el backup en el pod logs, este dato se agrega en el campo de name en el values.yaml de backups y cambiamos el type a restore. Tras hacer esto, podemos reinstalar el helm chart con: **helm uninstall backups helm install backups backups**

Cuando está en modo restore, no crea un CronJob, solo ejecuta un Job. Por lo tanto, podemos ver el progreso del Job en la pestaña de Jobs en Lens: Al final, podemos saber que el Job terminó cuando la columna de Completions salga en 1/1. Ahora mediante el mismo llamado en Postman: **GET**

http://127.0.0.1:50209/pokemon/_all_docs?include_docs=true

Y con el response se puede observar que el restore ha sido exitoso.

Neo4j

Para instalar Neo4j, se utilizó el Helm Chart oficial de Neo4j. Este se puede acceder de <https://helm.neo4j.com/neo4j>. Para añadir el repositorio, hay que hacer

```
helm repo add neo4j https://helm.neo4j.com/neo4j
```

Los Helm Charts disponibles de Neo4j son los siguientes.

neo4j/neo4j	5.13.0	5.13.0	Neo4j is the world's leading graph database
neo4j/neo4j-admin	5.13.0	5.13.0	Neo4j is the world's leading graph database
neo4j/neo4j-headless-service	5.13.0	-	Neo4j is the world's leading graph database
neo4j/neo4j-persistent-volume	5.13.0	-	Sets up persistent disks suitable for a Neo4j H...
neo4j/neo4j-reverse-proxy	5.13.0	5.13.0	Sets up an http server and a reverse proxy for ...

Para instalar la base de datos en sí, se tiene que añadir la dependencia al Chart.yaml del chart de databases.

```
- name: neo4j
  version: "5.12.0"
  repository: https://helm.neo4j.com/neo4j
  condition: neo4j.enabled
```

En los values.yaml, hay que poner los siguientes valores.

```
neo4j:
  enabled: true
neo4j:
  name: my-standalone
  resources:
    cpu: "0.5"
    memory: "2Gi"
  password: "neo4j-password"
  edition: "enterprise"
  acceptLicenseAgreement: "yes"
  config:
    server.directories.plugins: "/var/lib/neo4j/labs"
    dbms.security.procedures.unrestricted:
      "apoc.export.*,apoc.import.*,apoc.*,apoc.export"
    server.config.strict_validation.enabled: "false"
    dbms.security.procedures.allowlist: "apoc.export.*,apoc.import.*"
  apoc_config:
    apoc.trigger.enabled: "true"
    apoc.jdbc.neo4j.url: "jdbc:foo:bar"
```

```
apoc.import.file.enabled: "true"
apoc.export.file.enabled: "true"
env:
  NEO4J_PLUGINS: '["apoc"]'
volumes:
  data:
    mode: defaultStorageClass
    defaultStorageClass:
      requests:
        storage: 2Gi
```

Aquí hay varios parámetros importantes. La base de datos siempre debe tener un nombre, el parámetro de name. En este caso es my-standalone. El password es la contraseña del usuario neo4j, que es el usuario predeterminado. Si este campo se omite, la contraseña se genera automáticamente. Los resources definen el tamaño de la instancia de Neo4j. El mínimo es de 0.5 CPU y 2 Gb de memoria. El volumes.data asigna el volumen del contenedor que va a contener los datos. Este puede persistir aun cuando la base de datos no está instalada.

La sección de config se utiliza para instalar plugins, que son como métodos, funciones y procedimientos adicionales a los que ya tiene Neo4j. En este caso, estamos intentando de instalar el plugin de APOC Core. Este permite exportar e importar una base de datos con un archivo del dump. Se incluyen estos procedimientos y la configuración del plugin. Finalmente, está el parámetro más importante para nuestro caso de backup: el edition. Esto es debido a que los backups solo están disponibles en el Enterprise Edition de Neo4j.

Back up and restore (online)

Enterprise Edition

Backup

Enterprise Edition

Default port: 6362

Table 3. Backup

Related configuration setting	Default value	Description
<code>server.backup.listen_address</code>	127.0.0.1:6362	Network interface and port for the backup server to listen on.
<code>server.backup.enabled</code>	true	Enable support for running online backups.

In production environments, external access to the backup port should be blocked by a firewall.

Para levantar el pod de Neo4j, simplemente hay que hacer

```
helm install databases databases
```

Hay varias formas de hacer un backup en Neo4j. La primera es la que se utilizó para esta tarea. Para hacer un backup desde un job y luego subir el backup al bucket de AWS, se necesita buscar una forma instalar Neo4j en Amazon Linux. Esto es porque el pod del job usa una imagen del Amazon CLI. Para hacer esto, se utilizan las siguientes instrucciones:

```
rpm --import https://debian.neo4j.com/neotechnology.gpg.key
cat << EOF > /etc/yum.repos.d/neo4j.repo
[neo4j]
name=Neo4j RPM Repository
baseurl=https://yum.neo4j.com/stable/5
enabled=1
gpgcheck=1
EOF
NEO4J_ACCEPT_LICENSE_AGREEMENT=yes yum install neo4j-enterprise-5.13.0 -y
```

Es importante recalcar que hay que instalar la versión de Enterprise. La Community Edition de Neo4j no tiene la instrucción específica para hacer el backup. La instalación de Neo4j en Amazon Linux contiene las herramientas para interactuar con esta: neo4j y neo-admin. Se utiliza neo4j-admin para hacer el backup. La instrucción es la siguiente:

```
neo4j-admin database backup --from=$NEO4J_SERVICE:$NEO4J_PORT --type=full --to-path=/neo4j/$DATE
```

Este comando tiene varias opciones. Se explicarán a continuación las que se utilizaron en esta tarea:

- `--from=host:port` : Esta opción se utiliza para especificar la dirección de la base de datos de Neo4j. En nuestro caso, se necesita acceder al servicio del admin del pod de Neo4j. Esto se hace poniendo el nombre del pod-admin.default.svc.cluster.local. El puerto es el que se usa para los backups. En este caso es el 6362.
- `--type=full` : Se utiliza para especificar el tipo de backup que se necesita. Hay tres tipos: full, diff y auto. Un backup full contiene toda la base de datos. Un diferencial (diff) contiene una bitácora de las transacciones que se aplicaron a la base de datos desde el último full backup. Auto determina cuál backup hacer automáticamente.
- `--to-path=` : Especifica el directorio donde se va a almacenar el backup. Este puede ser en la misma base de datos, pero en este caso se utilizó el camino absoluto al directorio del pod del job para que se almacene el backup localmente en este. Luego de ejecutar la instrucción, se obtiene el siguiente resultado.

```
s/neo4j/restore_metadata.cypher. To execute the file use cypher-shell command with parameter `neo4j`
2023-10-23 20:42:07.692+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Start recovering database 'neo4j'.
2023-10-23 20:42:07.754+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Finish recovering database 'neo4j', took 59ms.
Restore of database 'neo4j' from path='/neo4j/202310232022/neo4j-2023-10-23T20-25-10.backup' completed successfully.
bash-4.2# neo4j-admin database backup --from=databases-admin.default.svc.cluster.local:6362 --type=full --to-path=/ne
o4j/$DATE
2023-10-23 20:46:13.191+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Starting backup of database 'neo4j' from servers:
[databases-admin.default.svc.cluster.local:6362]
2023-10-23 20:46:13.620+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start backup of database 'neo4j'.
2023-10-23 20:46:13.622+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Using remote server databases-admin.default.svc.cl
uster.local:6362 for backup of database 'neo4j'.
2023-10-23 20:46:13.767+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start full backup of database 'neo4j'.
2023-10-23 20:46:13.843+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start receiving store files for database 'neo4j'.
2023-10-23 20:46:13.947+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Finished receiving store files for database 'neo4j
', took 102ms.
2023-10-23 20:46:13.963+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start receiving database 'neo4j' transactions from
29.
2023-10-23 20:46:13.982+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Finished receiving transactions for database 'neo4
j' at 28, took 18ms.
2023-10-23 20:46:14.219+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Finished full backup of database 'neo4j'. Download
ed from tx 28 to tx 28.
2023-10-23 20:46:14.226+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start recovering database 'neo4j'.
2023-10-23 20:46:14.244+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Finished recovering database 'neo4j', took 17ms.
2023-10-23 20:46:14.244+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Start creating artifact 'neo4j-2023-10-23T20-46-14
.backup' for database 'neo4j'.
2023-10-23 20:46:14.326+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Finished artifact creation 'neo4j-2023-10-23T20-46
-14.backup' for database 'neo4j', took 80ms.
2023-10-23 20:46:14.330+0000 INFO [c.n.b.v.b.BackupOutputMonitor] Backup of database 'neo4j' completed, took 708ms.
Backup command completed.
```

Las variables de \$NEO4J_SERVICE y \$NEO4J_PORT se envían por el values.yaml del job. Para la sección de Neo4j, los valores son los siguientes:

```
neo4j:
  enabled: true
  config:
    namespace: default
    host: databases-admin.default.svc.cluster.local
    port: "6362"
    bucketName: tec-ic4302-02-2023
    path: 2019073558/neo4j
    maxBackups: 3
    name: neo4j-2023-10-23T20-53-22.backup
    schedule: "0 */12 * * *"
    diskSize: 2
    storageClass: hostpath
    provider: aws
    type: backup
```

Aquí podemos ver cómo se envían el host (la dirección de la base de datos), el puerto (6362) y la información del path para subir el backup a AWS. Las instrucciones para subirlo a AWS son las mismas para las otras bases de datos:

```
aws s3 cp /neo4j/$DATE s3://$BUCKET_NAME/$BACKUP_PATH/ --recursive
aws s3 ls s3://$BUCKET_NAME/$BACKUP_PATH/
```

Luego de correr el backup y subirlo a AWS, podemos revisar si se subió correctamente:

```

PRE 200420157/
PRE 200420158/
PRE 2019073558/
PRE 2021022613/
PRE 2021436942/
PRE 2022437529/
2023-10-03 19:48:30      0 a
2023-10-23 05:28:46    6312 movies-2023-10-23T05-28-43.backup
2023-10-23 18:50:37    21943 neo4j-2023-10-23T18-50-37.backup
2023-10-03 23:38:11      178 yum-200420157
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023

```

La segunda opción para hacer el backup es utilizar el Helm Chart del neo4j-admin para subir el backup directamente a Neo4j. Para hacer esto, es necesario crear un nuevo values.yaml file específico para este. Este archivo tiene parámetros diferentes:

```

neo4j:
  image: "neo4j/helm-charts-backup"
  imageTag: "5.12.0"
  jobSchedule: "0 17 * * *"
  successfulJobsHistoryLimit: 3
  failedJobsHistoryLimit: 1
  backoffLimit: 3

backup:
  bucketName: "tec-ic4302-02-2023"
  databaseAdminServiceName: "databases-admin"
  database: "neo4j"
  cloudProvider: "aws"
  secretName: "aws-credential"
  secretKeyName: "credentials"
  consistencyCheck:
    enabled: true

```

Las credenciales de AWS están en un secret de Kubernetes. Este se establece en el helm chart de Bootstrap. La imagen que se va a usar para el pod es la de neo4j/helm-charts-backup. Esta contiene las instrucciones para hacer el backup y subirlo al bucket. Se pone el horario del cronjob que se crea, los límites de intentos, la información del bucket de AWS, las bases de datos que se van a respaldar y el consistency check. Este se asegura de que los datos sean consistentes antes de hacer el backup.

Luego, hay que instalar el Helm Chart de neo4j-admin. Esto se puede hacer con

```

helm install backup-name neo4j/neo4j-admin -f /direccion-del-archivo/backup-values.yaml

```

Esto crea un cronjob con la configuración específica en el values.yaml. Uno puede manualmente correr el cronjob. Es importante que la base de datos de Neo4j se del Enterprise Edition, porque si no, no va a estar

abierto el puerto 6362, que es el de los backups, por lo que el job falla.

```
2023/10/23 00:04:13 printing current directory /var/lib/neo4j
2023/10/23 00:04:13 Address := databases-admin.default.svc.cluster.local:6362
2023/10/23 00:04:34 connectivity cannot be established
output = nc: connect to databases-admin.default.svc.cluster.local (10.107.14.72) port 6362 (tcp) failed: Connection refused
err = exit status 1
```

Si el job no falla, ocurre lo siguiente:

```
neo4j backup-manual hggg-wgdrj default N/A N/A 0 job docker BestEffort 43s Succeeded
```

```
2023/10/23 18:50:18 printing current directory /var/lib/neo4j
2023/10/23 18:50:18 Address := databases-admin.default.svc.cluster.local:6362
2023/10/23 18:50:18 Connectivity established with database databases-admin.default.svc.cluster.local:6362
2023/10/23 18:50:18 Connectivity with S3 bucket "tec-ic4302-02-2023" established
2023/10/23 18:50:18 Address := databases-admin.default.svc.cluster.local:6362
2023/10/23 18:50:18 Printing backup flags [database backup --from-databases-admin.default.svc.cluster.local:6362 --include-metastore=all --keep-failed=false --parallel-recovery=false --type=all]
2023/10/23 18:50:18 Backup completed for database neo4j
2023/10/23 18:50:18 Backup file name is neo4j-2023-10-23T18-50-37.backup
2023/10/23 18:50:18 Starting upload of file /backups/neo4j-2023-10-23T18-50-37.backup
2023/10/23 18:50:18 File neo4j-2023-10-23T18-50-37.backup uploaded to s3 bucket tec-ic4302-02-2023
```

```
PRE 200420157/
PRE 200420158/
PRE 2019073558/
PRE 2021022613/
PRE 2021436942/
PRE 2022437529/
2023-10-03 19:48:30      0 a
2023-10-23 05:28:46    6312 movies-2023-10-23T05-28-43.backup
2023-10-23 18:50:37   21943 neo4j-2023-10-23T18-50-37.backup
2023-10-03 23:38:11     178 yum-200420157
bash-4.2# aws s3 ls s3://tec-ic4302-02-2023
```

Aquí podemos observar que el backup sí se subió correctamente, pero no se subió al directorio de nuestro grupo. No obstante, cuando intenté cambiar la dirección del bucket, daba error de que no encontraba el bucket.

```
2023/10/23 18:50:04 Address := databases-admin.default.svc.cluster.local:6362
2023/10/23 18:50:04 Connectivity established with database databases-admin.default.svc.cluster.local:6362
2023/10/23 18:50:04 Unable to connect to s3 bucket tec-ic4302-02-2023/2023073558/neo4j
Here's why: operation error S3: ListObjectsV2, https response error StatusCode: 403, RequestID: 708F72067F8F2A, hostID: 56e60f5d0c5b5c7f9f0826fa30912a07e03e98b4a93b7e8a0c429b11a9d0e
```

Por lo tanto, usamos la primera forma para que sí se subiera al bucket y dirección correcta.

La tercera forma de hacer un backup es de la forma que no es en línea, con un dump. Es muy similar a la primera forma, pero la base de datos tiene que estar detenida. Esto se puede hacer con la herramienta de Neo4j. El comando es neo4j stop. El problema con esto es que para detenerla, uno tiene que estar directamente con en el pod de la base de datos, con kubectl exec -it nombre del pod. Por lo tanto, esta forma no nos sirve para esta tarea. No obstante, el procedimiento es el siguiente. Cuando uno ya está en el pod, hay que hacer

```
neo4j stop
neo4j-admin database dump neo4j
```

El resultado es el siguiente.

```
neo4j@databases-0:~$ neo4j-admin database dump neo4j
2023-10-24 19:17:37.805+0000 INFO [c.n.c.d.EnterpriseDumpCommand] Starting dump of database 'neo4j'
Done: 76 files, 258.1MiB processed.
2023-10-24 19:18:28.694+0000 INFO [c.n.c.d.EnterpriseDumpCommand] Dump completed successfully
neo4j@databases-0:~$
```

Para encontrar el dump, hay que entrar al directorio data/dumps/

```
neo4j@databases-0:~$ ls
LICENSE.txt  NOTICE.txt  UPGRADE.txt  certificates  data  labs  licenses  packaging_info  products
LICENSES.txt  README.txt  bin          conf          import  lib  logs      plugins         run
neo4j@databases-0:~$ cd data
neo4j@databases-0:~/data$ ls
cluster-state  databases  dbms  dumps  server_id  transactions
neo4j@databases-0:~/data$ cd dumps
neo4j@databases-0:~/data/dumps$ ls
neo4j.dump
```

La última forma de hacer un backup es por medio del plugin APOC. Este significa Awesome Procedures on Cypher. Este tiene unos procedimientos que son export e import. Hay varias versiones de export. Uno puede exportar a un archivo GraphML, Cypher, CSV, Excel, JSON, etc. Este comando se puede correr desde el browser de Neo4j.

file	batches	source	format	nodes	relationships	properties	time	rows	batchSize	cypherStatements	nodeStatements	relationshipStatements
"backup cypher"	1	"database: nodes(171), rels(253)"	"cypher"	171	253	564	313	424	20000	null	null	null

El resultado de esta instrucción se debería guardar en el directorio de imports del pod.

Para correr esta instrucción desde el pod del job, se puede utilizar el API de Neo4j. Se puede instalar curl y realizar un post al endpoint. Se utiliza el servicio y el puerto, como al igual que en la primera forma del backup. La instrucción para hacer esto es el siguiente. Se utiliza un > para guiar el resultado del query a un archivo del pod.

```
curl --verbose POST http://databases-
admin.default.svc.cluster.local:7474/db/neo4j/tx/commit -H "Content-
Type:application/json" -d "{\"statements\": [{\"statement\": \"CALL
apoc.export.cypher.all('backup.cypher', {useTypes: TRUE, storeNodeIds:
FALSE})\"}]}\" -H "Authorization: Basic bmVvNGo6bmVvNGotcGFzc3dvcmQ=" >
/neo4j/$DATE/backup.cypher
```

Sin embargo, como se ve en el resultado del query más arriba, el resultado no es el archivo en sí, es como un mensaje de éxito. Esta respuesta es lo que se guarda en el archivo, no el archivo del backup. Por lo tanto, esta opción no sirve para esta tarea.

Restore

Para hacer un restore, es muy similar a la primera forma de hacer el backup. Los values.yaml para el job son muy similares:

```
neo4j:
  enabled: true
  config:
    namespace: default
    host: databases-admin.default.svc.cluster.local
    port: "6362"
```

```

bucketName: tec-ic4302-02-2023
path: 2019073558/neo4j
maxBackups: 3
name: neo4j-2023-10-23T20-53-22.backup
schedule: "0 */12 * * *"
diskSize: 2
storageClass: hostpath
provider: aws
type: restore

```

Cuando uno le pone el type restore, eso indica que el job que se tiene que crear es el de restore y no el backup. Hay que instalar Neo4j Enterprise en el pod. Primero, hay que copiar el archivo del backup del bucket al pod. Esto se hace con la siguiente instrucción:

```
aws s3 cp s3://$BUCKET_NAME/$BACKUP_PATH/$RESTORE_FILE $DIRECTORY
```

Luego, se realiza la instrucción propia del restore. Este es muy similar al backup

```
neo4j-admin database restore --from-path=$DIRECTORY/$RESTORE_FILE --to-path-
data=$NEO4J_SERVICE:$NEO4J_PORT --verbose
```

Se envía el servicio de Neo4j y el puerto y ahí se realiza el restore.

```

2023-10-23 20:42:07.285+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 72/77, data: 90.3%
2023-10-23 20:42:07.286+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 91.0%
2023-10-23 20:42:07.287+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 91.8%
2023-10-23 20:42:07.287+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 92.5%
2023-10-23 20:42:07.288+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 93.2%
2023-10-23 20:42:07.288+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 93.9%
2023-10-23 20:42:07.288+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 73/77, data: 93.9%
2023-10-23 20:42:07.289+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 74/77, data: 94.7%
2023-10-23 20:42:07.290+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 74/77, data: 94.7%
2023-10-23 20:42:07.293+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 75/77, data: 94.9%
2023-10-23 20:42:07.295+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 76/77, data: 95.6%
2023-10-23 20:42:07.295+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 76/77, data: 95.6%
2023-10-23 20:42:07.296+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 96.4%
2023-10-23 20:42:07.296+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 97.1%
2023-10-23 20:42:07.297+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 97.8%
2023-10-23 20:42:07.297+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 98.5%
2023-10-23 20:42:07.298+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 99.3%
2023-10-23 20:42:07.299+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 100.0%
2023-10-23 20:42:07.300+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Files: 77/77, data: 100.0%
2023-10-23 20:42:07.303+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Done: 77 files, 1.074MiB processed.
2023-10-23 20:42:07.658+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Restoring from unpacked backup artifact chain:
/neo4j/202310232022/temp10000459030506007157
2023-10-23 20:42:07.691+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] You need to execute /var/lib/neo4j/data/script
s/neo4j/restore_metadata.cypher. To execute the file use cypher-shell command with parameter 'neo4j'
2023-10-23 20:42:07.692+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Start recovering database 'neo4j'.
2023-10-23 20:42:07.754+0000 INFO [c.n.b.v.r.RestoreDatabaseExecutor] Finish recovering database 'neo4j', took 59ms.
Restore of database 'neo4j' from path '/neo4j/202310232022/neo4j-2023-10-23T20-25-10.backup' completed successfully.
bash-4.2#

```

Así se debería ver la base de datos luego del restore.



Conclusiones

1. El uso de almacenamiento en la nube con AWS S3 facilita la escalabilidad y accesibilidad de los backups, pero requiere tomar en cuenta consideraciones adicionales de seguridad a la hora de hacer el código y mantener su control de versiones, como asegurarse que las credenciales de AWS estén siempre seguras y privadas.
2. Si bien Elasticsearch requiere una configuración inicial más elaborada y el proceso para habilitar snapshots es más manual, una vez que se tienen los componentes configurados adecuadamente, la automatización de snapshots y restores resulta bastante sencilla y más fácil de entender desde una perspectiva de usuario.
3. El uso de scripts de Linux resulta extremadamente útil para ejecutar automáticamente tareas más complejas en jobs o en un contenedor.
4. Las operaciones de backup y restore se pueden realizar de forma más fácil si la base de datos tiene herramientas de línea de comandos para esto y que además se pueden conectar a otro servidor por medio de parámetros de conexión. Por esta razón, los jobs de PostgreSQL, MariaDB y MongoDB resultaron más sencillas.
5. Los backups y restores son fundamentales para la recuperación de accidentes o fallas, como cuando se borra un registro por accidente o cuando se inicia un nuevo servidor y se quiere restaurarlo para que esté igual a otra base de datos. Para esto, son especialmente útiles los backups completos.
6. El cronjob es una herramienta útil para realizar los procesos de backup de una manera automática y segura.
7. Kubernetes es una herramienta esencial que permite realizar una amplia cantidad de pruebas e instalaciones de bases de datos.
8. Los helm charts continúan siendo una herramienta necesaria para la automatización del despliegue de las bases de datos.
9. El servicio ClusterIP que levantan las bases de datos resulta fundamental y extremadamente útil para conectarse a la base de datos desde otro pod en el cluster de Kubernetes.

10. El API de CouchDB nos facilita el manejo de los datos, al momento de hacer pruebas con herramientas como Postman o de hacer los backups de los documentos de manera eficiente.

Recomendaciones

1. Para implementar los backups desarrollados en este proyecto en un escenario real, se podrían implementar junto con herramientas de monitoreo como Prometheus y Grafana. Contar con dashboards personalizados en Grafana facilitaría identificar tendencias, bottlenecks y problemas en los procesos de backup.
2. Para establecer backups en Elasticsearch, se recomienda hacer uso de la interfaz interactiva de Kibana para crear políticas de snapshots. Esto en lugar de ejecutar comandos manuales sobre la base Elasticsearch para hacer cada backup de forma individual, ya que provee una experiencia más amigable e intuitiva para los usuarios.
3. En lugar de tener las credenciales de acceso a la nube y el bucket hardcoded en los scripts de backup, se recomienda almacenar estas credenciales como secrets de Kubernetes. De esta manera se centraliza la gestión de credenciales sensibles y se evita exponerlas en el código.
4. Es crítico que el repositorio de código donde se almacena la tarea de backup se mantenga privado y con acceso restringido. De esta manera se evita que personas no deseadas puedan ver el código fuente y obtener las credenciales que dan acceso a la cuenta de AWS u otra plataforma en la nube.
5. Para optimizar el uso de almacenamiento en AWS, se recomienda hacer uso de una política de retención de backups automatizada. En lugar de retener todos los backups por tiempo indefinido, se recomienda usar una política que conserve los backups más recientes y necesarios, y elimine versiones antiguas que ya no son relevantes.
6. Es crítico validar periódicamente que al implementar backups, estos sean funcionales y permitan restaurar los datos correctamente. Por eso, al implementar componentes de backups se recomienda realizar pruebas regulares de restore a partir de los mismos para confirmar su correcto funcionamiento y capacidad de recuperación de información.
7. En una tarea de este tipo, dividir el trabajo es importante, pero también es importante que cada persona del equipo entienda su tarea. Se recomienda mantener una buena comunicación durante el desarrollo del proyecto y nunca quedarse con dudas.
8. A la hora de probar los backups generados, se recomienda hacer uso de un pod de depuración como el incluido en la solución de esta tarea. Dicho pod proporciona una terminal con conectividad directa al bucket de almacenamiento en AWS. Esto permite validar de forma efectiva que los respaldos se estén ejecutando correctamente y subiendo a la nube según lo esperado.
9. Cuando se quiera hacer pruebas sobre las bases de datos con API's se recomienda el uso de aplicaciones como Postman o Thunder Client ya que estos nos ayudan a hacer request HTTP de forma sencilla.
10. A la hora de preparar el pod con el aws-cli, se recomienda buscar cómo instalar las herramientas necesarias para hacer el backup específicamente para la plataforma de Amazon Linux. Incluso, puede ser que Amazon tenga una implementación en el repositorio de extras para Amazon Linux, como ocurrió con PostgreSQL.

Referencias

Accessing Neo4J - Operations Manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/kubernetes/accessing-neo4j/>

Awesome Procedures on Cypher (APOC) - Neo4J Labs. (s. f.). Neo4j Graph Data Platform. [https://neo4j.com/labs/apoc/#:~:text=APOC%20\(Awesome%20Procedures%20on%20Cypher,a%20lot%20of%20useful%20functionality.](https://neo4j.com/labs/apoc/#:~:text=APOC%20(Awesome%20Procedures%20on%20Cypher,a%20lot%20of%20useful%20functionality.)

Back up an offline database - operations manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/backup-restore/offline-backup/>

Back up an online database - operations manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/backup-restore/online-backup/>

Back up and restore (online) - Operations manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/kubernetes/operations/backup-restore/>

Create a Helm Deployment Values.yaml file - Operations Manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/kubernetes/quickstart-standalone/create-value-file/>

Dump and Load Databases (offline) - Operations Manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/kubernetes/operations/dump-load/>

Export - APOC extended documentation. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/labs/apoc/4.1/export/>

Red Hat, CentOS, Fedora, and Amazon Linux (.RPM) - Operations Manual. (s. f.). Neo4j Graph Data Platform. <https://neo4j.com/docs/operations-manual/current/installation/linux/rpm/>